

Alberta's Workforce & Economy Analysis

Data Analysis to Understand Key Labor and Economic Trends

Introduction

Alberta's economy has long been driven by key industries such as oil and gas, agriculture, manufacturing, construction, and technology. In an increasingly globalized world, international events and market trends influence economic outcomes, often introducing unexpected shifts and volatility. This project analyzes trends in total employment, GDP, employment by gender, and average weekly earnings across different industries in Alberta. By examining these factors, we aim to better understand how macroeconomic forces shape the province's workforce and overall economic landscape. Employment data, including industry-wise workforce counts and breakdowns by age group and gender, have been used to explore labor force trends. Average weekly earnings across industries have been analyzed to identify wage differences, and GDP data have been studied to understand each sector's economic contribution. Additionally, unemployment rates have been assessed to determine job stability and economic resilience within different industries. Long-term patterns, the effects of economic shocks due to pandemics, and shifts in workforce distribution have also been identified.

Several benefits are offered by this analysis. Industries that require support or investment can be identified by policymakers, while businesses can be assisted in understanding wage trends and workforce availability. Job seekers can also be guided toward sectors with stable employment opportunities. By examining earnings, GDP, and unemployment rates together, an assessment can be made regarding whether wage growth aligns with economic expansion and how different industries have been affected by job market fluctuations. Additionally, workforce participation shifts can be highlighted by tracking labor force trends by age and gender, which may inform policies on training and workforce development. Overall, valuable insights into Alberta's evolving economy and labor market are provided by this study, supporting better decision-making for the future.

Importing Libraries

```
In [3]: import pandas as pd
        from sqlalchemy import create_engine, text
        import sqlalchemy as sq
        import matplotlib.pyplot as plt
        from sqlalchemy import inspect
        import numpy as np
        import plotly.express as px
        from scipy import stats
        import seaborn as sns
```

Datasets

Data Source: Statistics Canada

- Labour force characteristics by industry, annual (x 1,000):
<https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410002301>
- Average weekly earnings by industry, monthly, unadjusted for seasonality
<https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410020301&pickMembers%5B0%5D=1.2&pickMembers%5B1%5D=2.1&pickMem>
- Employment by industry, monthly, unadjusted for seasonality
<https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410020101&pickMembers%5B0%5D=1.10&pickMembers%5B1%5D=2.1&cubeTim>
- Gross domestic product (GDP) at basic prices, by industry, provinces and territories (x 1,000,000) <https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=3610040201&pickMembers%5B0%5D=2.1&pickMembers%5B1%5D=3.1&cubeTime>

Individual Datasets

```
In [4]: labourForce= pd.read_csv('Labour force characteristics by industry, annual.csv')
display(labourForce.head())
```

	REF_DATE	GEO	Labour force characteristics	North American Industry Classification System (NAICS)	Gender	Age group	VALUE
0	1976	Canada	Labour force	Total, all industries	Total - Gender	15 years and over	10491.3
1	1976	Canada	Labour force	Total, all industries	Total - Gender	15 to 24 years	2891.9
2	1976	Canada	Labour force	Total, all industries	Total - Gender	25 to 54 years	6410.7
3	1976	Canada	Labour force	Total, all industries	Total - Gender	55 years and over	1188.7
4	1976	Canada	Labour force	Total, all industries	Men+	15 years and over	6549.6

```
In [5]: avgEarning = pd.read_csv('Average weekly earnings by industry, monthly, unadjusted')
display(avgEarning.head())
```

	REF_DATE	GEO	Type of employees	Overtime	North American Industry Classification System (NAICS)	VALUE
0	2001-01	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	669.59
1	2001-02	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	669.55
2	2001-03	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	667.45
3	2001-04	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	670.74
4	2001-05	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	663.49

```
In [6]: employmentData = pd.read_csv("Employment by industry, monthly, unadjusted for seasonality")
display(employmentData.head())
```

	REF_DATE	GEO	Type of employee	North American Industry Classification System (NAICS)	VALUE
0	2001-01	Alberta	All employees	Industrial aggregate including unclassified bu...	1362768.0
1	2001-02	Alberta	All employees	Industrial aggregate including unclassified bu...	1384194.0
2	2001-03	Alberta	All employees	Industrial aggregate including unclassified bu...	1388464.0
3	2001-04	Alberta	All employees	Industrial aggregate including unclassified bu...	1402013.0
4	2001-05	Alberta	All employees	Industrial aggregate including unclassified bu...	1433384.0

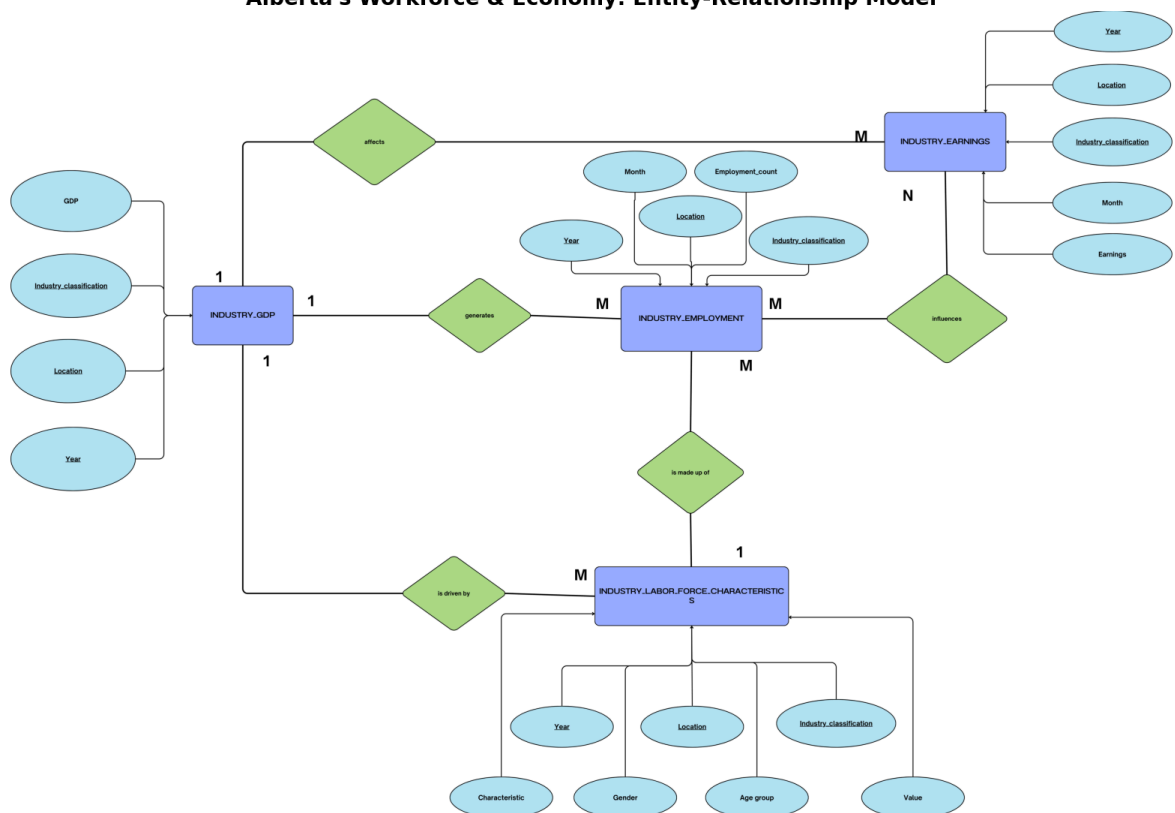
```
In [7]: gdp = pd.read_csv("Gross domestic product (GDP) at basic prices, by industry, province")
display(gdp.head())
```

	REF_DATE	GEO	Value	North American Industry Classification System (NAICS)	VALUE
0	1997	Alberta	Current dollars	All industries [T001]	104192.9
1	1997	Alberta	Current dollars	Goods-producing industries [T002]	43283.3
2	1997	Alberta	Current dollars	Service-producing industries [T003]	60909.6
3	1997	Alberta	Current dollars	Industrial production [T010]	33876.5
4	1997	Alberta	Current dollars	Non-durable manufacturing industries [T011]	6204.5

```
In [8]: import matplotlib.pyplot as plt
import matplotlib.image as mpimg

image_path = "ER_diagram.png"
img = mpimg.imread(image_path)
plt.figure(figsize=(20, 14))
plt.title("Alberta's Workforce & Economy: Entity-Relationship Model", fontsize=20)
plt.imshow(img)
plt.axis('off')
plt.show()
```

Alberta's Workforce & Economy: Entity-Relationship Model



Guiding Questions

1. How does gender-based employment vary across different industries?
2. Which industries experienced the highest percentage growth in female employment between 2001 and 2024?
3. Are certain industries dominated by specific age groups?
4. What is the distribution of full-time vs. part-time employment across industries?
5. How has the proportion of male and female employment changed over time?
6. Which are the top 5 industries with the highest and lowest average weekly earnings?
7. What is the overall distribution of average weekly earnings across different industries?
8. What are the average weekly earnings and average employment for each industry?
9. Which industries are biggest and least in terms of providing employment opportunities?
10. Which industries in Alberta lost the most jobs during the COVID period (March 2020 to December 2021)?
11. Did industries with higher GDP before COVID suffer less during the pandemic?
12. Which industries have seen the highest growth in GDP, and how do they compare in employment levels/average earnings?

Data Exploration

Connect to Database

```
In [9]: engine = sq.create_engine('mysql+mysqlconnector://project:SWMPnkbZdwDmL@localhost:3306')  
conn = engine.connect()
```

Data Exploration - Part 1

Labour force characteristics by industry: Cleaning and Data analysis

Cleaning - Dataset Labour Force - Part 1

```
In [10]: #Cleaning column values of Industry Classification and Gender  
labourForce['North American Industry Classification System (NAICS)'] = labourForce['NAICS']
```

```
labourForce['Gender'] = labourForce['Gender'].str.replace(r'^a-zA-Z0-9 ', '',
```

Dataframe to SQL - Labour Force Data

```
In [11]: labourForce_sql=labourForce.to_sql('LabourForce_table',engine,if_exists='replace
#chunksize will break our rows into batches, which will help in managing memory
print("DataFrame successfully converted to SQL table!")
```

DataFrame successfully converted to SQL table!

Check column names of LabourForce table

```
In [13]: i = inspect(engine)
for column in i.get_columns('LabourForce_table'):
    print(f"{column['name']},{column['type']}")
```

REF_DATE,BIGINT

GEO,TEXT

Labour force characteristics,TEXT

North American Industry Classification System (NAICS),TEXT

Gender,TEXT

Age group,TEXT

VALUE,DOUBLE

Count of Labour force data, Unique values of columns like Type of Labour Force and Industry Classification

```
In [14]: labourForce_count=pd.read_sql("SELECT COUNT(*) FROM LabourForce_table;",engine)
display(labourForce_count)
labourForce_type=pd.read_sql('SELECT DISTINCT `Labour force characteristics` FRO
display(labourForce_type)
naics=pd.read_sql('SELECT DISTINCT `North American Industry Classification Syste
display(naics)
```

COUNT(*)

0	1021152
---	---------

Labour force characteristics

0	Labour force
---	--------------

1	Employment
---	------------

2	Full-time employment
---	----------------------

3	Part-time employment
---	----------------------

4	Unemployment
---	--------------

5	Unemployment rate
---	-------------------

North American Industry Classification System (NAICS)

0	Total, all industries
1	Goods-producing sector
2	Agriculture
3	Forestry, fishing, mining, quarrying, oil and gas
4	Utilities
5	Construction
6	Manufacturing
7	Services-producing sector
8	Wholesale and retail trade
9	Transportation and warehousing
10	Finance, insurance, real estate, rental and le...
11	Professional, scientific and technical services
12	Business, building and other support services
13	Educational services
14	Health care and social assistance
15	Information, culture and recreation
16	Accommodation and food services
17	Other services (except public administration)
18	Public administration
19	Unclassified industries
20	Forestry and logging and support activities fo...
21	Fishing, hunting and trapping
22	Mining, quarrying, and oil and gas extraction
23	Durables
24	Non-durables
25	Wholesale trade
26	Retail trade
27	Finance and insurance
28	Real estate and rental and leasing

Cleaning - Dataset Labour Force - Part 2

```
In [15]: cleaning_labourForce_Data = [  
        "DROP TABLE IF EXISTS years;",
```

```

"DROP TABLE IF EXISTS geo;",
"DROP TABLE IF EXISTS cleaned_LabourForce;",
"DROP TABLE IF EXISTS filtered_LabourForce;",
"DROP TABLE IF EXISTS final_LabourForce;",

""""CREATE TABLE years AS
SELECT * FROM LabourForce_table
WHERE CAST(REF_DATE AS INTEGER) BETWEEN 2001 AND 2024;""",

""""CREATE TABLE geo AS
SELECT * FROM years
WHERE TRIM(GEO) = 'Alberta';""",

""""CREATE TABLE cleaned_LabourForce AS
SELECT `REF_DATE` AS YEAR, `GEO` AS Province, Gender AS Gender,
`Labour force characteristics` AS `Characteristics`,
`North American Industry Classification System (NAICS)` AS `Industry`,
`Age group` AS Age_Group,
COALESCE(VALUE, 0) AS Value FROM geo;""",

""""CREATE TABLE filtered_LabourForce AS
SELECT * FROM cleaned_LabourForce WHERE
LOWER(TRIM(Industry)) != 'total, all industries'
AND LOWER(TRIM(Gender)) NOT LIKE '%total%gender%'
AND LOWER(TRIM(Age_Group)) != '15 years and over'
AND LOWER(TRIM(Characteristics)) != 'Employment';""",

""""CREATE TABLE final_LabourForce AS
SELECT YEAR, Province, Characteristics, Industry, Gender, Age_Group, Value
FROM filtered_LabourForce WHERE Industry NOT IN (
'Industrial aggregate excluding unclassified businesses',
'Goods-producing sector',
'Durables',
'Non-durables',
'Services-producing sector',
'Unclassified industries',
'Forestry, fishing, mining, quarrying, oil and gas',
'Fishing, hunting and trapping',
'Wholesale trade',
'Retail trade',
'Finance, insurance, real estate, rental and leasing');"""]

for query in cleaning_labourForce_Data:
    conn.execute(text(query))
print("The Labour Force dataset has been successfully cleaned!")

```

The Labour Force dataset has been successfully cleaned!

```

In [16]: final_Labourdata = pd.read_sql("SELECT* FROM final_LabourForce", engine)
         final_Labourdata.head()

```


Out[16]:

	YEAR	Province	Characteristics	Industry	Gender	Age_Group	Value
0	2001	Alberta	Labour force	Agriculture	Men	15 to 24 years	6.3
1	2001	Alberta	Labour force	Agriculture	Men	25 to 54 years	24.4
2	2001	Alberta	Labour force	Agriculture	Men	55 years and over	12.3
3	2001	Alberta	Labour force	Agriculture	Women	15 to 24 years	2.3
4	2001	Alberta	Labour force	Agriculture	Women	25 to 54 years	11.4

In [17]: `final_data_count = pd.read_sql("SELECT COUNT(*) FROM final_LabourForce", engine)`
`final_data_count`

Out[17]:

	COUNT(*)
0	12960

Guiding Questions - Part 1

1. How does gender-based employment vary across different industries?

In [18]:

```
genderParticipation= """SELECT Industry,Gender,SUM(Value) AS Total_Employment_in
FROM final_LabourForce GROUP BY Industry, Gender ORDER BY Total_Employment_in_th
genderParticipation=pd.read_sql(genderParticipation, engine)
display(genderParticipation.head())
display(genderParticipation.tail())
#visual
men=genderParticipation.loc[genderParticipation['Gender'] == 'Men', ['Industry',
women=genderParticipation.loc[genderParticipation['Gender'] == 'Women', ['Indust

classification=women['Industry'].values
men_values = men['Total_Employment_in_thousands'].values
women_values = women['Total_Employment_in_thousands'].values

x= np.arange(len(classification))
bar_width=0.40

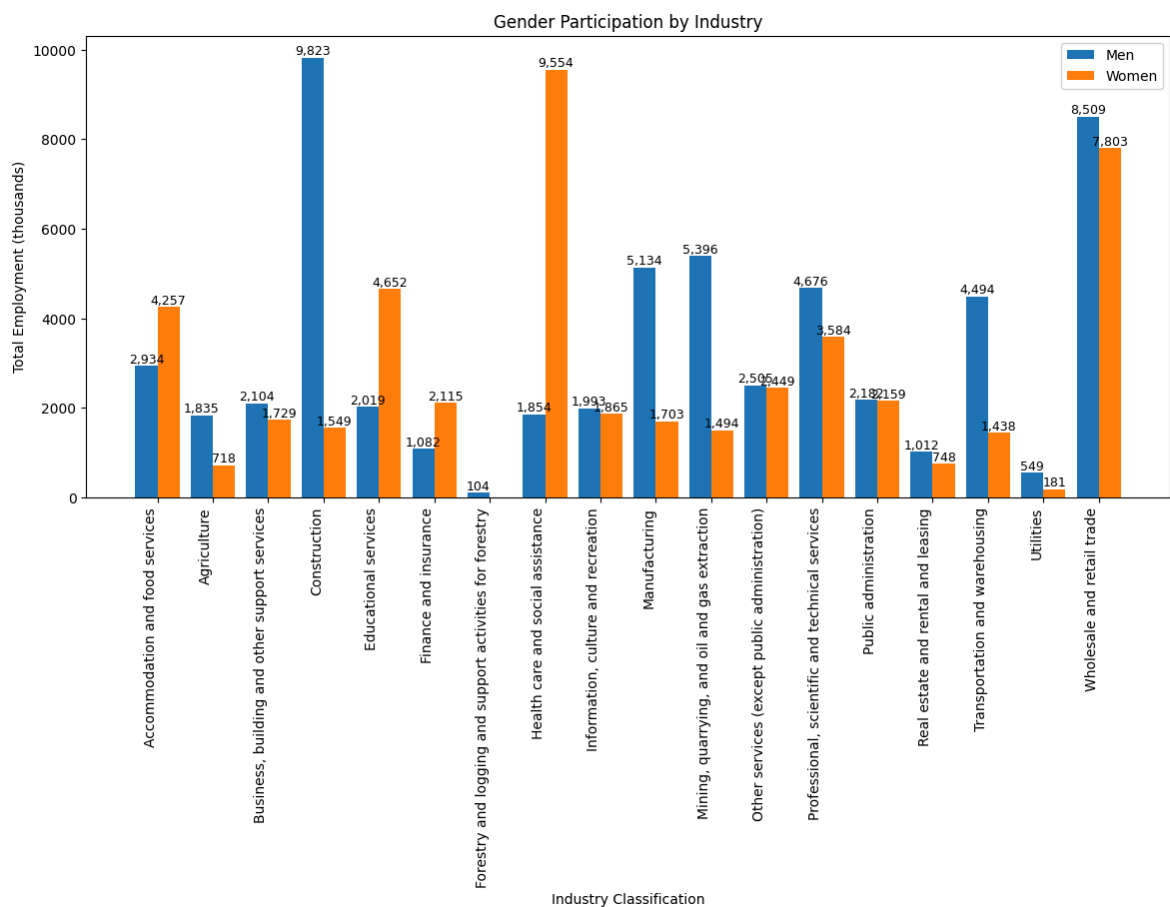
fig, ax = plt.subplots(figsize=(14, 6))
bar1=ax.bar(x - bar_width/2, men_values, width=bar_width, label='Men')
bar2=ax.bar(x + bar_width/2, women_values, width=bar_width, label='Women')
for bars in [bar1,bar2]:
    for bar in bars:
        height=bar.get_height()
        if height>0:
            ax.text(bar.get_x()+bar.get_width()/2,height,f'{height:,.0f}',ha="ce

plt.xticks(x,classification,rotation=90, ha='right')
plt.xlabel('Industry Classification')
plt.legend()
plt.ylabel('Total Employment (thousands)')
plt.title('Gender Participation by Industry')
```

	Industry	Gender	Total_Employment_in_thousands
0	Construction	Men	9822.8
1	Health care and social assistance	Women	9553.8
2	Wholesale and retail trade	Men	8509.2
3	Wholesale and retail trade	Women	7803.0
4	Mining, quarrying, and oil and gas extraction	Men	5396.1

	Industry	Gender	Total_Employment_in_thousands
31	Agriculture	Women	718.5
32	Utilities	Men	548.6
33	Utilities	Women	181.1
34	Forestry and logging and support activities fo...	Men	104.4
35	Forestry and logging and support activities fo...	Women	0.0

Out[18]: Text(0.5, 1.0, 'Gender Participation by Industry')



The way jobs are split between men and women varies a lot in different industries, with some fields having a much higher number of one gender than the other.

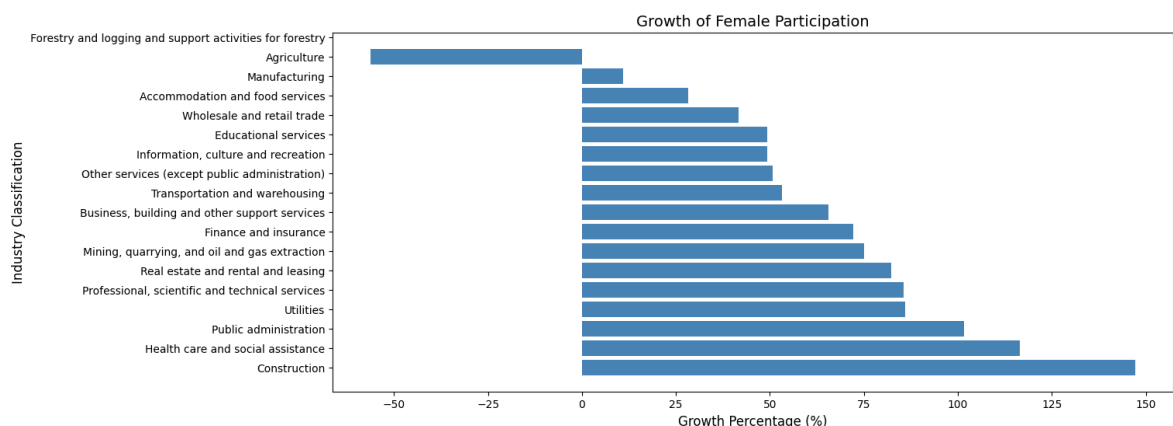
- Men dominate industries like *Construction* (9,822.8K), *Wholesale and retail trade* (8,509.2 K), *Mining, quarrying, and oil and gas extraction* (5,396.1K) while women lead in *Healthcare and social assistance* (9,553.8K) and *Wholesale and retail trade* (7,803 K).
- Although *wholesale and retail trade* utilize both sexes in comparatively higher numbers, the male-biased disparity persists (8,509.2 thousand men compared to 7,803 thousand women).

2. Which industries experienced the highest percentage growth in female employment between 2001 and 2024?

```
In [19]: femaleCount=pd.read_sql("""SELECT Industry, YEAR, SUM(Value) AS Female_Count
FROM final_LabourForce
WHERE Gender = 'Women' GROUP BY Industry, YEAR ORDER BY Industry, YEAR;""",engine)
earliest_year = femaleCount['YEAR'].min()
latest_year = femaleCount['YEAR'].max()
start = femaleCount[femaleCount['YEAR'] == earliest_year].set_index('Industry')[
end = femaleCount[femaleCount['YEAR'] == latest_year].set_index('Industry')['Female_Count']
percentage_growth = ((end - start)/start) * 100
growth_value = percentage_growth.sort_values(ascending=False).to_frame(name='Percentage_Growth')
display(growth_value)

#visual
plt.figure(figsize=(14, 6))
plt.barh(growth_value.index, growth_value['Percentage Growth (%)'], color='steelblue')
plt.title('Growth of Female Participation', fontsize=14)
plt.xlabel('Growth Percentage (%)', fontsize=12)
plt.ylabel('Industry Classification', fontsize=12)
plt.show()
```

	Percentage Growth (%)
Industry	
Construction	147.147147
Health care and social assistance	116.554721
Public administration	101.628664
Utilities	86.000000
Professional, scientific and technical services	85.556578
Real estate and rental and leasing	82.291667
Mining, quarrying, and oil and gas extraction	75.000000
Finance and insurance	72.246066
Business, building and other support services	65.631068
Transportation and warehousing	53.153153
Other services (except public administration)	50.742240
Information, culture and recreation	49.354376
Educational services	49.204328
Wholesale and retail trade	41.650890
Accommodation and food services	28.276780
Manufacturing	10.926694
Agriculture	-56.151420
Forestry and logging and support activities for forestry	NaN



The highest growth in female employment (2001-2024) was in *Construction* (147.15%), *Health care* (116.55%), and *Public administration* (101.63%).

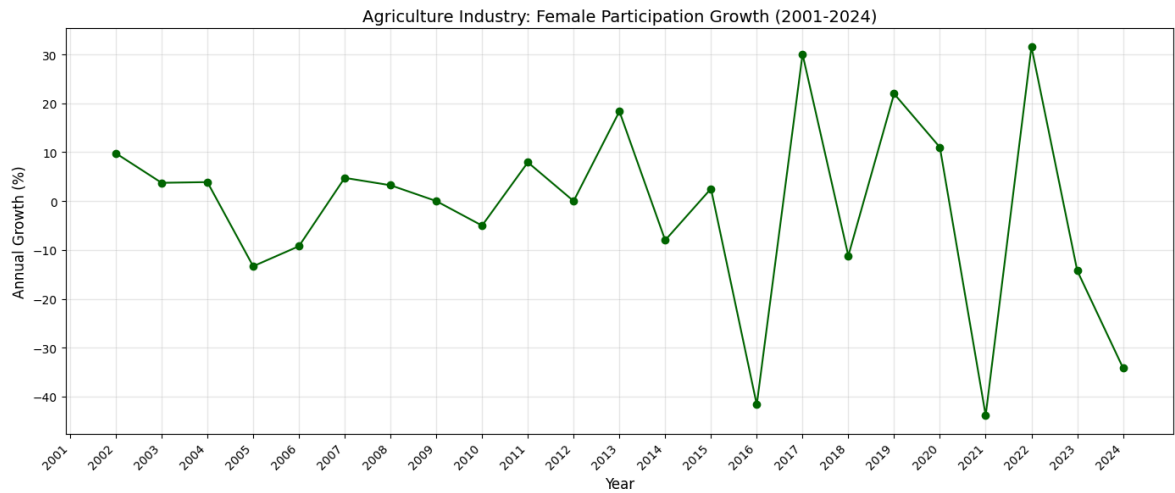
2.1. Why is female participation in agriculture far lower than in other female-led industries?

```
In [20]: agricultureFemale = femaleCount[femaleCount['Industry'] == 'Agriculture'].copy()

#how much female participation increased/decreased each year compared to the pre
agricultureFemale['Growth_Percentage'] = agricultureFemale.groupby('Industry')['
display(agricultureFemale[['YEAR', 'Growth_Percentage']])

plt.figure(figsize=(14, 6))
plt.plot(agricultureFemale['YEAR'], agricultureFemale['Growth_Percentage'],
         marker='o', color='darkgreen')
plt.xticks(ticks=agricultureFemale['YEAR'], rotation=45, ha='right', fontsize=10)
plt.title('Agriculture Industry: Female Participation Growth (2001-2024)', fontsize=10)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Annual Growth (%)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

	YEAR	Growth_Percentage
24	2001	NaN
25	2002	9.779180e+00
26	2003	3.735632e+00
27	2004	3.878116e+00
28	2005	-1.333333e+01
29	2006	-9.230769e+00
30	2007	4.745763e+00
31	2008	3.236246e+00
32	2009	-1.110223e-14
33	2010	-5.015674e+00
34	2011	7.920792e+00
35	2012	0.000000e+00
36	2013	1.834862e+01
37	2014	-8.010336e+00
38	2015	2.528090e+00
39	2016	-4.164384e+01
40	2017	3.004695e+01
41	2018	-1.119134e+01
42	2019	2.195122e+01
43	2020	1.100000e+01
44	2021	-4.384384e+01
45	2022	3.155080e+01
46	2023	-1.422764e+01
47	2024	-3.412322e+01



A few years report sharp declines (2005, 2016, 2021), while others report sharp increases (2013, 2017, 2023), indicating irregular female labor force participation. These fluctuations suggest that external factors such as economic conditions, technological advancements, and policy changes significantly impact women's employment in agriculture.

3. Are certain industries dominated by specific age groups?

```
In [21]: age=pd.read_sql("""SELECT Industry, Age_Group, SUM(Value)
AS Total_Value_in_thousands FROM final_LabourForce
GROUP BY Industry, Age_Group
ORDER BY Total_Value_in_thousands DESC;""", engine)
#display(age)

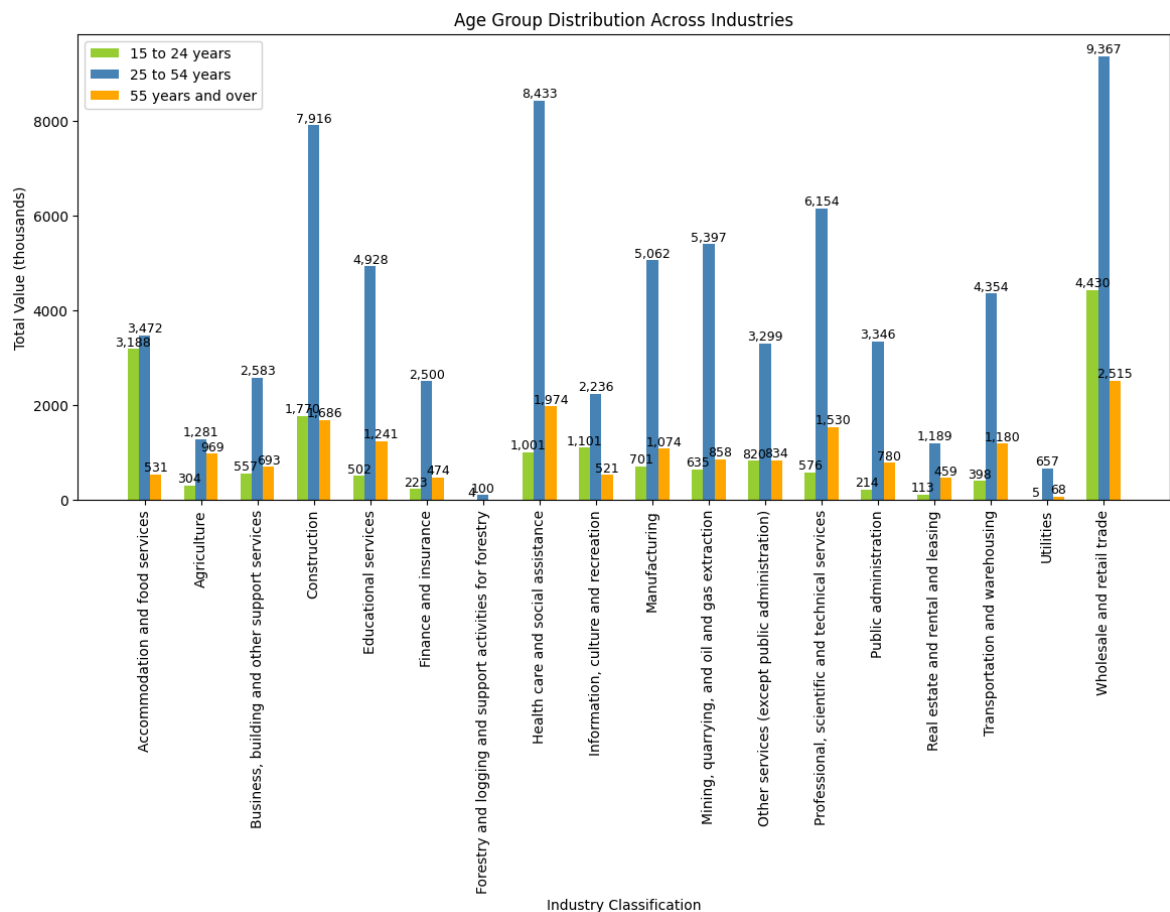
group1=age.loc[age['Age_Group'] == '15 to 24 years', ['Industry', 'Total_Value_in
group2=age.loc[age['Age_Group'] == '25 to 54 years', ['Industry', 'Total_Value_in
group3=age.loc[age['Age_Group']=='55 years and over', ['Industry', 'Total_Value_in

classification=group1['Industry'].values
group_1 = group1['Total_Value_in_thousands'].values
group_2 = group2['Total_Value_in_thousands'].values
group_3= group3['Total_Value_in_thousands'].values

x= np.arange(len(classification))
bar_width=0.2

fig, ax = plt.subplots(figsize=(14, 6))
bar1 = ax.bar(x - bar_width, group_1, width=bar_width, label='15 to 24 years', c
bar2 = ax.bar(x, group_2, width=bar_width, label='25 to 54 years', color='steelb
bar3 = ax.bar(x + bar_width, group_3, width=bar_width, label='55 years and over'
for bars in [bar1, bar2, bar3]:
    for bar in bars:
        height = bar.get_height()
        if height > 0:
            ax.text(bar.get_x() + bar.get_width()/2, height, f'{height:,.0f}',
                    ha='center', va='bottom', fontsize=9, color='black')

plt.xticks(x, classification, rotation=90)
plt.xlabel('Industry Classification')
plt.ylabel('Total Value (thousands)')
plt.legend(loc='upper left')
plt.title('Age Group Distribution Across Industries')
plt.show()
```



```
In [22]: highestIndustry = age.groupby('Age_Group').head(1)
display(highestIndustry)
lowestIndustry = age.groupby('Age_Group').tail(1)
display(lowestIndustry)
```

	Industry	Age_Group	Total_Value_in_thousands
0	Wholesale and retail trade	25 to 54 years	9367.4
7	Wholesale and retail trade	15 to 24 years	4430.0
14	Wholesale and retail trade	55 years and over	2514.8

	Industry	Age_Group	Total_Value_in_thousands
49	Forestry and logging and support activities fo...	25 to 54 years	100.4
52	Forestry and logging and support activities fo...	15 to 24 years	4.0
53	Forestry and logging and support activities fo...	55 years and over	0.0

- Most of the workers are in the age group of 25 to 54 years. Followed by 15 to 24 years and 55+
- Highest number of workers in each age groups works in the industry *Wholesale and retail trade*

- Lower number of workers in the industry *Forestry and logging and support activities for forestry*
-

4. What is the distribution of full-time vs. part-time employment across industries?

```
In [23]: characteristics=pd.read_sql("""SELECT Industry,Characteristics,SUM(Value)
AS Total_Value_in_thousands FROM final_LabourForce
WHERE Characteristics IN('Full-time employment','Part-time employment')
GROUP BY Industry,Characteristics ORDER BY Total_Value_in_thousands DESC ;""",en
display(characteristics.head())
display(characteristics.tail())
plt.figure(figsize=(12, 6))
full_employ=characteristics.loc[characteristics['Characteristics'] == 'Full-time
part_employ=characteristics.loc[characteristics['Characteristics'] == 'Part-time

classification=full_employ['Industry'].values
full_values = full_employ['Total_Value_in_thousands'].values
part_values = part_employ['Total_Value_in_thousands'].values

x= np.arange(len(classification))
bar_width=0.30

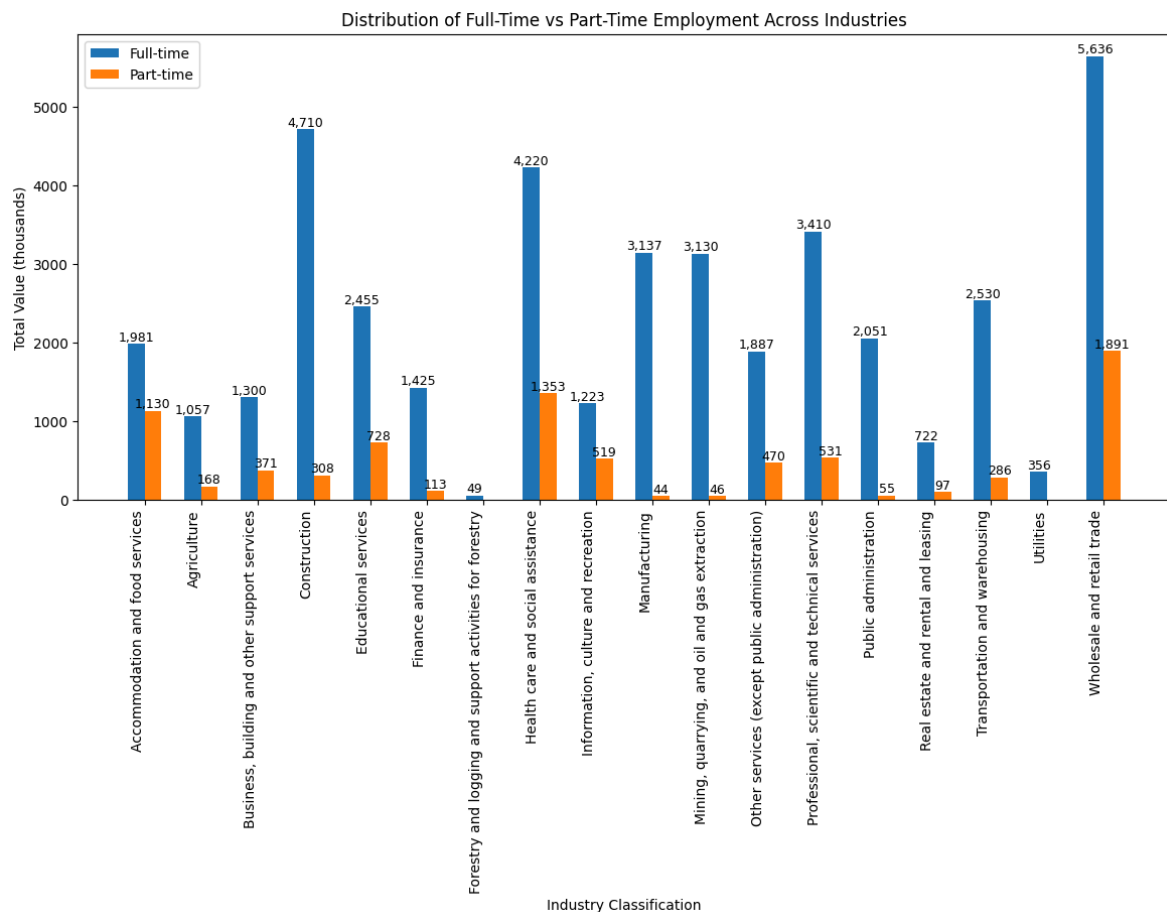
fig, ax = plt.subplots(figsize=(14, 6))
bar1 = ax.bar(x - bar_width/2, full_values, width=bar_width, label='Full-time')
bar2 = ax.bar(x + bar_width/2, part_values, width=bar_width, label='Part-time')
for bars in [bar1, bar2]:
    for bar in bars:
        height = bar.get_height()
        if height > 0:
            ax.text(bar.get_x() + bar.get_width()/2, height, f'{height:,.0f}',
                    ha='center', va='bottom', fontsize=9, color='black')

plt.xticks(x,classification,rotation=90, ha='right')
plt.xlabel('Industry Classification')
plt.ylabel('Total Value (thousands)')
plt.legend()
plt.title('Distribution of Full-Time vs Part-Time Employment Across Industries')
plt.show()
```

	Industry	Characteristics	Total_Value_in_thousands
0	Wholesale and retail trade	Full-time employment	5636.5
1	Construction	Full-time employment	4710.2
2	Health care and social assistance	Full-time employment	4219.5
3	Professional, scientific and technical services	Full-time employment	3410.0
4	Manufacturing	Full-time employment	3136.7

	Industry	Characteristics	Total_Value_in_thousands
31	Forestry and logging and support activities fo...	Full-time employment	49.4
32	Mining, quarrying, and oil and gas extraction	Part-time employment	46.4
33	Manufacturing	Part-time employment	44.2
34	Forestry and logging and support activities fo...	Part-time employment	0.0
35	Utilities	Part-time employment	0.0

<Figure size 1200x600 with 0 Axes>



- *Wholesale and retail trade* (5,636.5k), *Construction* (4,710.2k), *Health care and social assistance* (4,219.5k) have highest **Full-time employment**
- *Wholesale and retail trade* (1,890.8k) *Health care and social assistance* (1,352.6k), *Accommodation and food services* (1,129.6k) lead in **Part-time employment**
- Notably, *Utilities* and *Forestry and logging and support activities for forestry* have **no Part-time employment**

5. How has the proportion of male and female employment changed over time?

```
In [24]: genderTrend=pd.read_sql("""SELECT Year,Gender,SUM(Value) AS Workforce_Count_in_t
FROM final_LabourForce WHERE Gender IN ('Men', 'Women')
GROUP BY Year, Gender ORDER BY Year, Workforce_Count_in_thousands DESC;""",engin
display(genderTrend)

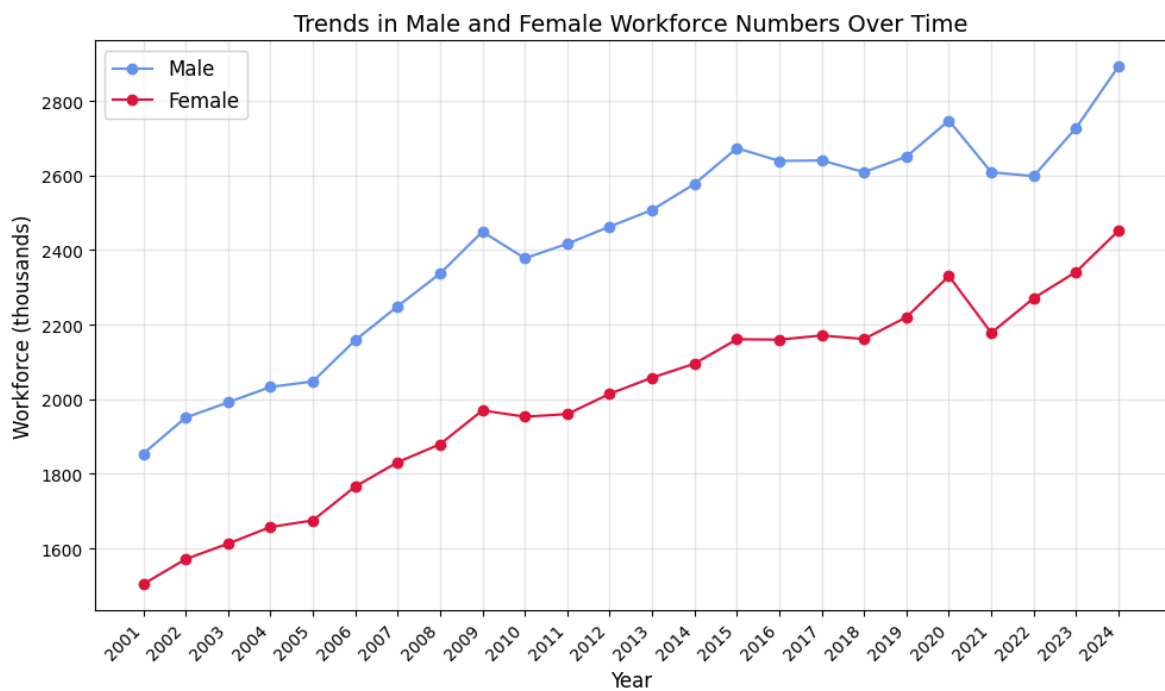
plt.figure(figsize=(10, 6))
male = genderTrend.loc[genderTrend['Gender'] == 'Men', ['Year', 'Workforce_Count
female = genderTrend.loc[genderTrend['Gender'] == 'Women', ['Year', 'Workforce_C
plt.plot(male['Year'], male['Workforce_Count_in_thousands'],
         marker='o', label='Male', color='cornflowerblue', linestyle='-')
plt.plot(female['Year'], female['Workforce_Count_in_thousands'],
         marker='o', label='Female', color='crimson', linestyle='-')

plt.xticks(ticks=male['Year'],rotation=45,ha='right',fontsize=10)
```

```
plt.legend(fontsize=12)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Workforce (thousands)', fontsize=12)
plt.title('Trends in Male and Female Workforce Numbers Over Time', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

	Year	Gender	Workforce_Count_in_thousands
0	2001	Men	1854.1
1	2001	Women	1504.2
2	2002	Men	1950.8
3	2002	Women	1571.3
4	2003	Men	1991.6
5	2003	Women	1612.8
6	2004	Men	2033.1
7	2004	Women	1657.5
8	2005	Men	2047.8
9	2005	Women	1675.1
10	2006	Men	2158.3
11	2006	Women	1766.0
12	2007	Men	2249.4
13	2007	Women	1831.2
14	2008	Men	2337.1
15	2008	Women	1879.4
16	2009	Men	2449.5
17	2009	Women	1970.2
18	2010	Men	2378.1
19	2010	Women	1953.2
20	2011	Men	2417.0
21	2011	Women	1960.3
22	2012	Men	2463.2
23	2012	Women	2014.6
24	2013	Men	2507.7
25	2013	Women	2058.0
26	2014	Men	2577.5
27	2014	Women	2095.4
28	2015	Men	2673.5
29	2015	Women	2160.9
30	2016	Men	2639.1
31	2016	Women	2160.0
32	2017	Men	2640.6

	Year	Gender	Workforce_Count_in_thousands
33	2017	Women	2171.1
34	2018	Men	2609.2
35	2018	Women	2161.6
36	2019	Men	2650.8
37	2019	Women	2220.4
38	2020	Men	2747.6
39	2020	Women	2330.6
40	2021	Men	2608.9
41	2021	Women	2178.2
42	2022	Men	2598.4
43	2022	Women	2271.2
44	2023	Men	2728.0
45	2023	Women	2341.7
46	2024	Men	2892.8
47	2024	Women	2452.3



It is observed that there is no consistent decrease or increase in the gender gap.

5.1. How has the employment gender gap varied across the analysis period for industries with the highest male and female employment?

```
In [25]: genderDiff="""SELECT YEAR,Industry, SUM(CASE WHEN Gender = 'Men' THEN Value ELSE
SUM(CASE WHEN Gender = 'Women' THEN Value ELSE 0 END) AS WomenCount,
(SUM(CASE WHEN Gender = 'Men' THEN Value ELSE 0 END) - SUM(CASE WHEN Gender = 'W
FROM final_LabourForce WHERE Characteristics IN ('Full-time employment','Part-ti
GROUP BY YEAR, Industry ORDER BY YEAR, Industry;"""
genderDiff=pd.read_sql(genderDiff, engine)
display(genderDiff)
```

	YEAR	Industry	MenCount	WomenCount	Difference
0	2001	Accommodation and food services	44.3	65.2	-20.9
1	2001	Agriculture	41.4	13.9	27.5
2	2001	Business, building and other support services	19.4	23.7	-4.3
3	2001	Construction	112.0	15.6	96.4
4	2001	Educational services	31.3	74.7	-43.4
...	...	...	...	...	...
427	2024	Public administration	57.4	60.0	-2.6
428	2024	Real estate and rental and leasing	27.2	17.4	9.8
429	2024	Transportation and warehousing	109.9	33.3	76.6
430	2024	Utilities	14.3	4.4	9.9
431	2024	Wholesale and retail trade	186.4	167.4	19.0

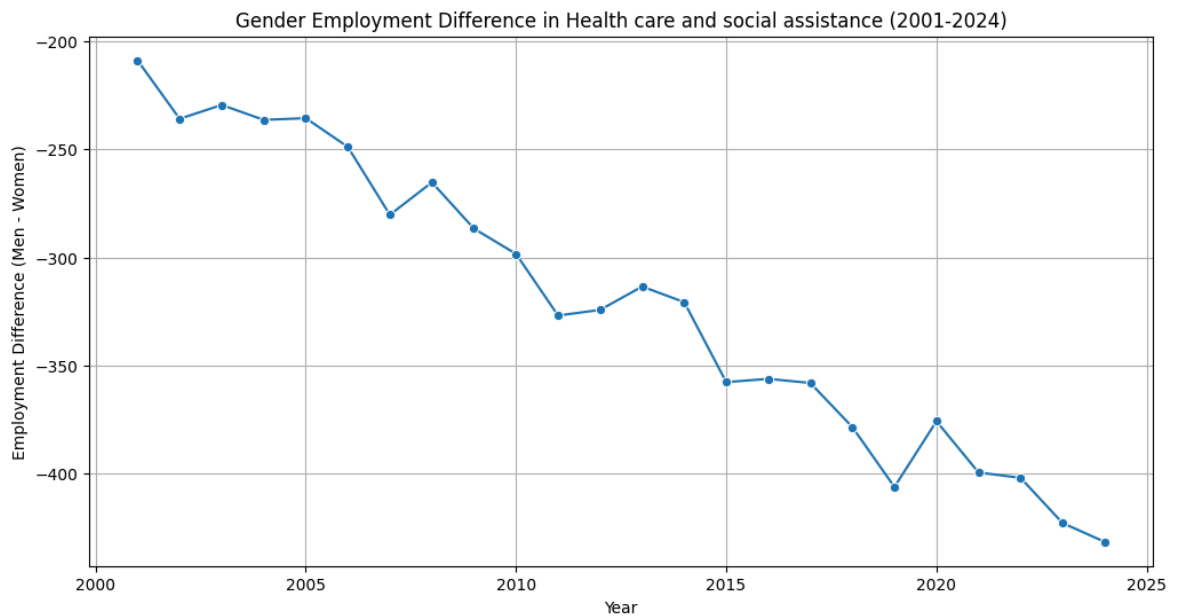
432 rows × 5 columns

```
In [26]: health_female_diff="""SELECT YEAR,Industry, SUM(CASE WHEN Gender = 'Men' THEN Va
SUM(CASE WHEN Gender = 'Women' THEN Value ELSE 0 END) AS WomenCount,
(SUM(CASE WHEN Gender = 'Men' THEN Value ELSE 0 END) - SUM(CASE WHEN Gender = 'W
FROM final_LabourForce WHERE Industry = 'Health care and social assistance'
GROUP BY YEAR, Industry ORDER BY YEAR, Industry;"""
health_female_diff=pd.read_sql(health_female_diff, engine)
display(health_female_diff)

health_female_diff_visual = health_female_diff[(health_female_diff["Industry"] =
(health_female_diff["YEAR"].between(2001, 2024))

plt.figure(figsize=(12, 6))
sns.lineplot(data=health_female_diff_visual, x="YEAR", y="Difference", marker="c
plt.title("Gender Employment Difference in Health care and social assistance (20
plt.xlabel("Year")
plt.ylabel("Employment Difference (Men - Women)")
plt.grid(True)
plt.show()
```

	YEAR	Industry	MenCount	WomenCount	Difference
0	2001	Health care and social assistance	44.2	253.1	-208.9
1	2002	Health care and social assistance	39.8	275.7	-235.9
2	2003	Health care and social assistance	45.0	274.5	-229.5
3	2004	Health care and social assistance	49.8	286.2	-236.4
4	2005	Health care and social assistance	58.6	294.2	-235.6
5	2006	Health care and social assistance	58.2	307.1	-248.9
6	2007	Health care and social assistance	48.1	328.4	-280.3
7	2008	Health care and social assistance	61.0	326.3	-265.3
8	2009	Health care and social assistance	59.8	346.5	-286.7
9	2010	Health care and social assistance	69.2	367.6	-298.4
10	2011	Health care and social assistance	67.9	394.8	-326.9
11	2012	Health care and social assistance	72.0	396.3	-324.3
12	2013	Health care and social assistance	77.5	391.0	-313.5
13	2014	Health care and social assistance	86.7	407.4	-320.7
14	2015	Health care and social assistance	85.2	442.9	-357.7
15	2016	Health care and social assistance	86.3	442.5	-356.2
16	2017	Health care and social assistance	91.5	449.6	-358.1
17	2018	Health care and social assistance	91.6	470.1	-378.5
18	2019	Health care and social assistance	85.3	491.5	-406.2
19	2020	Health care and social assistance	116.8	492.6	-375.8
20	2021	Health care and social assistance	110.8	510.3	-399.5
21	2022	Health care and social assistance	116.8	518.8	-402.0
22	2023	Health care and social assistance	115.4	538.3	-422.9
23	2024	Health care and social assistance	116.5	548.1	-431.6

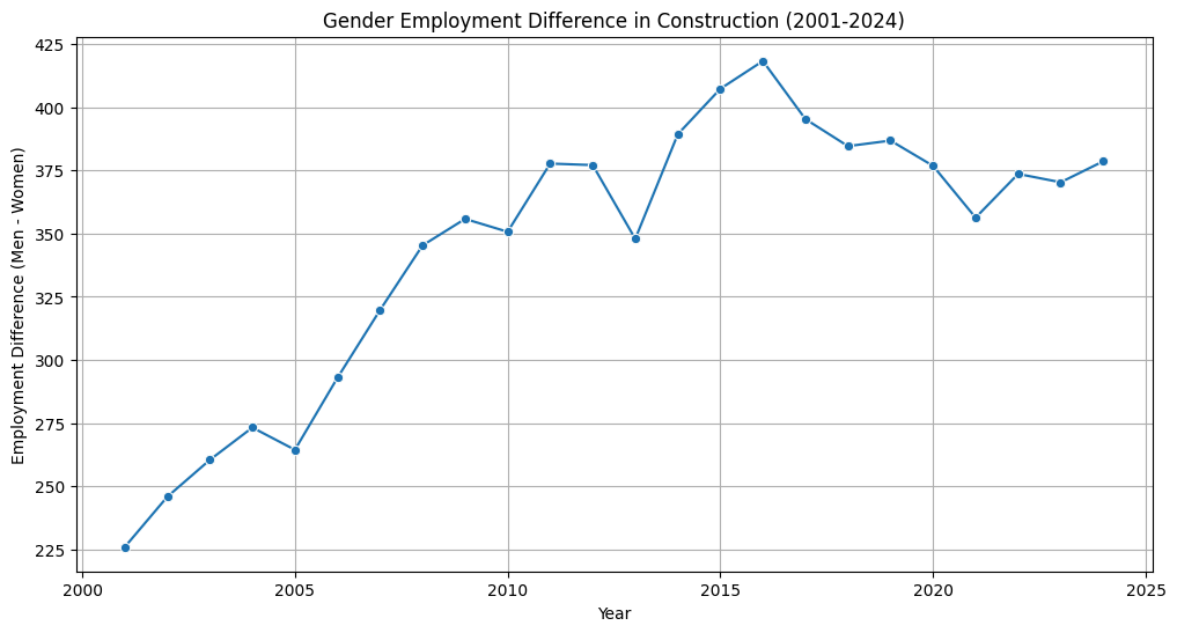


Health care and social assistance is the industry in which women outnumber men by the widest margin. Although the number of both genders is increasing, the gender gap continues to widen.

```
In [27]: construction_male_diff="""SELECT YEAR,Industry, SUM(CASE WHEN Gender = 'Men' THEN Value ELSE 0 END) AS MenCount,
SUM(CASE WHEN Gender = 'Women' THEN Value ELSE 0 END) AS WomenCount,
(SUM(CASE WHEN Gender = 'Men' THEN Value ELSE 0 END) - SUM(CASE WHEN Gender = 'Women' THEN Value ELSE 0 END)) AS Difference
FROM final_LabourForce WHERE Industry = 'Construction'
GROUP BY YEAR, Industry ORDER BY YEAR, Industry;"""
construction_male_diff=pd.read_sql(construction_male_diff, engine)
display(construction_male_diff)

construction_male_diff_visual = construction_male_diff[(construction_male_diff["Industry"] == 'Construction')]
plt.figure(figsize=(12, 6))
sns.lineplot(data=construction_male_diff_visual, x="YEAR", y="Difference", marker='o')
plt.title("Gender Employment Difference in Construction (2001-2024)")
plt.xlabel("Year")
plt.ylabel("Employment Difference (Men - Women)")
plt.grid(True)
plt.show()
```

	YEAR	Industry	MenCount	WomenCount	Difference
0	2001	Construction	259.2	33.3	225.9
1	2002	Construction	278.6	32.8	245.8
2	2003	Construction	293.1	32.7	260.4
3	2004	Construction	313.5	40.3	273.2
4	2005	Construction	307.1	42.7	264.4
5	2006	Construction	342.6	49.7	292.9
6	2007	Construction	376.4	56.7	319.7
7	2008	Construction	405.8	60.5	345.3
8	2009	Construction	419.4	63.6	355.8
9	2010	Construction	410.5	59.8	350.7
10	2011	Construction	436.6	58.9	377.7
11	2012	Construction	446.2	69.1	377.1
12	2013	Construction	432.0	84.1	347.9
13	2014	Construction	474.5	85.2	389.3
14	2015	Construction	492.0	84.7	407.3
15	2016	Construction	493.0	74.7	418.3
16	2017	Construction	477.1	81.8	395.3
17	2018	Construction	460.1	75.5	384.6
18	2019	Construction	464.6	77.8	386.8
19	2020	Construction	464.5	87.7	376.8
20	2021	Construction	430.2	73.9	356.3
21	2022	Construction	442.6	69.0	373.6
22	2023	Construction	442.3	72.0	370.3
23	2024	Construction	460.9	82.3	378.6



Construction is the industry in which men outnumber women by the widest margin. This gap widened until 2016, then saw a slight reduction until 2021, after which it has been increasing again.

Data Exploration - Part 2

Employment: Cleaning

```
In [28]: employmentData_sql=employmentData.to_sql(name='employment_data', if_exists='replace',
print("DataFrame successfully converted to SQL table!")
```

DataFrame successfully converted to SQL table!

```
In [29]: renaming_query= [
# Renaming columns
'ALTER TABLE employment_data RENAME COLUMN `GEO` TO `Province`;',
'ALTER TABLE employment_data RENAME COLUMN `Type of employee` TO `Employee_Type`;',
'ALTER TABLE employment_data RENAME COLUMN `North American Industry Classification` TO `Industry`;'
]
for query in renaming_query:
conn.execute(text(query))
```

```
In [30]: regex_query=[
# Cleaning Industry column by replacing "[" to "]" with "" using REGEXP
"""UPDATE employment_data
SET Industry = REGEXP_REPLACE(Industry, '\\\\[.?!\\\\]', '')
WHERE Industry REGEXP '\\\\[.?!\\\\]';""",
]
for query in regex_query:
conn.execute(text(query))
```

```
In [31]: # Removing Goods producing industries, Service producing industries, Industrial ag
classification_filterData=[
```

```

"DROP TABLE IF EXISTS employment_data_filtered;",
""""CREATE TABLE employment_data_filtered AS
SELECT REF_DATE,Province,CASE
WHEN `Industry` LIKE 'Forestry, logging and support%'
THEN 'Forestry and logging and support activities for forestry'
WHEN `Industry` LIKE 'Trade%'
THEN 'Wholesale and retail trade' ELSE `Industry` END AS `Industry`,VALUE
FROM employment_data
WHERE `Industry` NOT IN (
    "Goods producing industries",
    "Service producing industries",
    "Industrial aggregate excluding unclassified businesses");"""]
for query in classification_filterData:
    conn.execute(text(query))

new_industries_data = pd.read_sql("SELECT DISTINCT Industry FROM employment_data
display(new_industries_data)

```

	Industry
0	Industrial aggregate including unclassified bu...
1	Forestry and logging and support activities fo...
2	Mining, quarrying, and oil and gas extraction
3	Utilities
4	Construction
5	Manufacturing
6	Wholesale and retail trade
7	Transportation and warehousing
8	Information and cultural industries
9	Finance and insurance
10	Real estate and rental and leasing
11	Professional, scientific and technical services
12	Management of companies and enterprises
13	Administrative and support, waste management a...
14	Educational services
15	Health care and social assistance
16	Arts, entertainment and recreation
17	Accommodation and food services
18	Other services (except public administration)
19	Public administration
20	Unclassified businesses

```
In [32]: # Splitting year and month in table
date_formatted_query = """SELECT
    REF_DATE,
    YEAR(STR_TO_DATE(REF_DATE, '%Y-%m')) AS "YEAR",
    MONTH(STR_TO_DATE(REF_DATE, '%Y-%m')) AS "MONTH",
    Province,
    Industry,
    VALUE
FROM
    employment_data_filtered;"""
date_formatted_data = pd.read_sql(date_formatted_query, engine)
display(date_formatted_data)
```

	REF_DATE	YEAR	MONTH	Province	Industry	VALUE
0	2001-01	2001	1	Alberta	Industrial aggregate including unclassified bu...	1362768.0
1	2001-02	2001	2	Alberta	Industrial aggregate including unclassified bu...	1384194.0
2	2001-03	2001	3	Alberta	Industrial aggregate including unclassified bu...	1388464.0
3	2001-04	2001	4	Alberta	Industrial aggregate including unclassified bu...	1402013.0
4	2001-05	2001	5	Alberta	Industrial aggregate including unclassified bu...	1433384.0
...	...	...	...	...	...	...
6043	2024-08	2024	8	Alberta	Unclassified businesses	48705.0
6044	2024-09	2024	9	Alberta	Unclassified businesses	50475.0
6045	2024-10	2024	10	Alberta	Unclassified businesses	50721.0
6046	2024-11	2024	11	Alberta	Unclassified businesses	51446.0
6047	2024-12	2024	12	Alberta	Unclassified businesses	53262.0

6048 rows × 6 columns

```
In [33]: date_formatted_data.to_sql('employment_data',engine,if_exists='replace', index=F
```

Out[33]: 6048

```
In [34]: #Aggregation of

# 1. "Information and cultural industries" and "Arts, entertainment and recreati
# 2. "Management of companies and enterprises" and "Administrative and support,

#Change names of Information and cultural industries and Arts, entertainment and
# Management of companies and enterprises and Administrative and support, waste

renaming_industry_categories_query = """UPDATE employment_data
    SET Industry =
    CASE
    WHEN Industry = "Information and cultural i
```

```

        WHEN Industry = "Arts, entertainment and re
        WHEN Industry = "Management of companies an
        WHEN Industry = "Administrative and support
        END
        WHERE Industry IN ("Information and cultur
        "Arts, entertainment and recreation",
        "Management of companies and enterprises",
        "Administrative and support, waste manageme

conn.execute(text(renaming_industry_categories_query))
conn.commit()
new_industries_data = pd.read_sql("SELECT DISTINCT Industry FROM employment_data
display(new_industries_data)

```

	Industry
0	Industrial aggregate including unclassified bu...
1	Forestry and logging and support activities fo...
2	Mining, quarrying, and oil and gas extraction
3	Utilities
4	Construction
5	Manufacturing
6	Wholesale and retail trade
7	Transportation and warehousing
8	Information, culture and recreation
9	Finance and insurance
10	Real estate and rental and leasing
11	Professional, scientific and technical services
12	Business, building and other support services
13	Educational services
14	Health care and social assistance
15	Accommodation and food services
16	Other services (except public administration)
17	Public administration
18	Unclassified businesses

```

In [35]: # Creating Final table
# Consolidated dataframe
emp_count_final_tab_create_query = [
    "DROP TABLE IF EXISTS Employment_Count;",
    """
        CREATE TABLE Employment_Count AS
        SELECT REF_DATE,
        YEAR,
        MONTH,

```

```

        Province,
        Industry,
        CASE
            WHEN Industry IN (
                'Business, building and other support services',
                'Information, culture and recreation')
            THEN SUM(COALESCE(CAST(VALUE AS FLOAT), 0))
            ELSE COALESCE(CAST(VALUE AS FLOAT), 0)
        END AS VALUE
    FROM
    employment_data
    GROUP BY
    REF_DATE,
    YEAR,
    MONTH,
    Province,
    Industry;"""]

for query in emp_count_final_tab_create_query:
    conn.execute(text(query))
conn.commit()
# Checking final table
check_table_query = "SELECT * FROM Employment_Count;"
table_data = pd.read_sql(check_table_query,engine)
display(table_data.head())

```

	REF_DATE	YEAR	MONTH	Province	Industry	VALUE
0	2001-01	2001	1	Alberta	Accommodation and food services	119202.0
1	2001-01	2001	1	Alberta	Business, building and other support services	65531.0
2	2001-01	2001	1	Alberta	Construction	80249.0
3	2001-01	2001	1	Alberta	Educational services	100098.0
4	2001-01	2001	1	Alberta	Finance and insurance	0.0

```

In [36]: # Renaming VALUE column
rename_query = "ALTER TABLE Employment_Count RENAME COLUMN `VALUE` TO `Employment_Count`"
conn.execute(text(rename_query))
conn.commit()

```

```

In [37]: #Converting Employment table to CSV file
check_table_query = "SELECT * FROM Employment_Count;"
table_data = pd.read_sql(check_table_query,engine)
table_data.to_csv("Employment_Count_clean.csv",index=False)

```

Data Exploration - Part 3

Average Weekly Earnings: Cleaning

```

In [38]: # Cleaning industry column
avgEarning['North American Industry Classification System (NAICS)'] = avgEarning

```

```
avgEarning.head()
```

Out[38]:

	REF_DATE	GEO	Type of employees	Overtime	North American Industry Classification System (NAICS)	VALUE
0	2001-01	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	669.59
1	2001-02	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	669.55
2	2001-03	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	667.45
3	2001-04	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	670.74
4	2001-05	Alberta	All employees	Including overtime	Industrial aggregate excluding unclassified bu...	663.49

```
In [39]: avgEarning_sql=avgEarning.to_sql('AverageWeekly',engine,if_exists='replace', ind
```

```
In [40]: # Removing Goods producing industries,Service producing industries,Industrial ag
classification_filterData = [
    "DROP TABLE IF EXISTS AverageWeekly_filter;",
    """"CREATE TABLE AverageWeekly_filter AS
    SELECT REF_DATE,GEO,
        CASE
            WHEN `North American Industry Classification System (NAICS)` LIKE 'F
            THEN 'Forestry and logging and support activities for forestry'
            WHEN `North American Industry Classification System (NAICS)` LIKE 'T
            THEN 'Wholesale and retail trade'
            ELSE `North American Industry Classification System (NAICS)`
        END AS `North American Industry Classification System (NAICS)`,
        VALUE
    FROM AverageWeekly
    WHERE `North American Industry Classification System (NAICS)` NOT IN (
        "Goods producing industries",
        "Service producing industries",
        "Industrial aggregate excluding unclassified businesses"
    );""""
]

for query in classification_filterData:
    conn.execute(text(query))
classification_filterData = pd.read_sql( 'SELECT DISTINCT `North American Indust
classification_filterData
```


Out[40]: **North American Industry Classification System (NAICS)**

0	Forestry and logging and support activities fo...
1	Mining, quarrying, and oil and gas extraction
2	Utilities
3	Construction
4	Manufacturing
5	Wholesale and retail trade
6	Transportation and warehousing
7	Information and cultural industries
8	Finance and insurance
9	Real estate and rental and leasing
10	Professional, scientific and technical services
11	Management of companies and enterprises
12	Administrative and support, waste management a...
13	Educational services
14	Health care and social assistance
15	Arts, entertainment and recreation
16	Accommodation and food services
17	Other services (except public administration)
18	Public administration

```
In [41]: # Renaming columns
rename_queries = [
    'ALTER TABLE AverageWeekly_filter RENAME COLUMN `GEO` TO `Province`;',
    'ALTER TABLE AverageWeekly_filter RENAME COLUMN `North American Industry Cla
]

for query in rename_queries:
    conn.execute(text(query))
```

```
In [42]: # Splitting year and month in table
date_formatted_query = """SELECT
    REF_DATE,
    YEAR(STR_TO_DATE(REF_DATE, '%Y-%m')) AS "YEAR",
    MONTH(STR_TO_DATE(REF_DATE, '%Y-%m')) AS "MONTH",
    Province,
    Industry,
    VALUE
FROM
    AverageWeekly_filter;"""
date_formatted_data = pd.read_sql(date_formatted_query, engine)
```

```
In [43]: # Writing new df into the table
date_formatted_data.to_sql('AverageWeekly',engine,if_exists='replace', index=False)

# Display the new table
avg_weekly_data_query = "SELECT * FROM AverageWeekly;"
avg_weekly_data = pd.read_sql_query(avg_weekly_data_query, engine)

display(avg_weekly_data.head())
```

	REF_DATE	YEAR	MONTH	Province	Industry	VALUE
0	2001-01	2001	1	Alberta	Forestry and logging and support activities fo...	786.78
1	2001-02	2001	2	Alberta	Forestry and logging and support activities fo...	780.47
2	2001-03	2001	3	Alberta	Forestry and logging and support activities fo...	782.70
3	2001-04	2001	4	Alberta	Forestry and logging and support activities fo...	782.24
4	2001-05	2001	5	Alberta	Forestry and logging and support activities fo...	784.92

```
In [44]: #Aggregation of

# 1. "Information and cultural industries" and "Arts, entertainment and recreati
# 2. "Management of companies and enterprises" and "Administrative and support,

#Change names of Information and cultural industries and Arts, entertainment and
# Management of companies and enterprises and Administrative and support, waste

renaming_industry_categories_query = """UPDATE AverageWeekly
SET Industry =
CASE
WHEN Industry = "Information and cultural i
WHEN Industry = "Arts, entertainment and re
WHEN Industry = "Management of companies an
WHEN Industry = "Administrative and support
END
WHERE Industry IN ("Information and cultur
"Arts, entertainment and recreation",
"Management of companies and enterprises",
"Administrative and support, waste manageme

conn.execute(text(renaming_industry_categories_query))
conn.commit()

# Display the new table to confirm the names are changed
new_industries_data = pd.read_sql("SELECT DISTINCT Industry FROM AverageWeekly;"
display(new_industries_data)
```

Industry	
0	Forestry and logging and support activities fo...
1	Mining, quarrying, and oil and gas extraction
2	Utilities
3	Construction
4	Manufacturing
5	Wholesale and retail trade
6	Transportation and warehousing
7	Information, culture and recreation
8	Finance and insurance
9	Real estate and rental and leasing
10	Professional, scientific and technical services
11	Business, building and other support services
12	Educational services
13	Health care and social assistance
14	Accommodation and food services
15	Other services (except public administration)
16	Public administration

```
In [45]: #Consolidation of subcategories
# Creating final table
avg_weekly_final_table_create_query = [
    "DROP TABLE IF EXISTS Avg_week_earnings;",
    ""CREATE TABLE Avg_week_earnings AS
    SELECT
    REF_DATE,
    YEAR,
    MONTH,
    Province,
    Industry,
    CASE
        WHEN Industry IN (
            'Business, building and other support services',
            'Information, culture and recreation')
        THEN AVG(COALESCE(CAST(VALUE AS FLOAT), 0))
        ELSE COALESCE(CAST(VALUE AS FLOAT), 0)
    END AS VALUE
    FROM
    AverageWeekly
    GROUP BY
    REF_DATE,
    YEAR,
    MONTH,
    Province,
    Industry;"""]
```

```

for query in avg_weekly_final_table_create_query:
    conn.execute(text(query))
conn.commit()
# Checking final table
check_table_query = "SELECT * FROM Avg_week_earnings;"
table_data = pd.read_sql(check_table_query,engine)

```

```

In [46]: # Renaming VALUE column
rename_query = "ALTER TABLE Avg_week_earnings RENAME COLUMN `VALUE` TO `Avg_Week"
conn.execute(text(rename_query))
conn.commit()
check_table_query = "SELECT * FROM Avg_week_earnings;"
table_data = pd.read_sql(check_table_query,engine)
display(table_data.head())

```

	REF_DATE	YEAR	MONTH	Province	Industry	Avg_Weekly_Earnings
0	2001-01	2001	1	Alberta	Accommodation and food services	273.799988
1	2001-01	2001	1	Alberta	Business, building and other support services	739.760010
2	2001-01	2001	1	Alberta	Construction	864.200012
3	2001-01	2001	1	Alberta	Educational services	638.469971
4	2001-01	2001	1	Alberta	Finance and insurance	0.000000

```

In [47]: #Converting Average Weekly Earnings table to CSV file
table_data.to_csv("avgweek_clean.csv",index=False)

```

Data Exploration - Part 4

GDP Data: Cleaning

```

In [48]: gdp.describe()

```

```

Out[48]:

```

	REF_DATE	VALUE
count	27297.000000	24102.000000
mean	2010.000000	4324.943691
std	7.789024	24042.595546
min	1997.000000	-7.846000
25%	2003.000000	0.023000
50%	2010.000000	142.600000
75%	2017.000000	1207.775000
max	2023.000000	366736.100000

```
In [49]: #Extracting relevant data and renaming columns
gdp = gdp[(gdp["Value"] == "Current dollars") & (gdp["REF_DATE"] >= 2000)]
gdp.loc[:, "North American Industry Classification System (NAICS)"] = gdp["North
gdp.rename(columns = {"North American Industry Classification System (NAICS)": "
```

```
In [50]: gdp_df_cleaned = gdp.copy()

# merge unique cases Agric;fishing and infor
classes_to_merge = [
    ["Forestry and logging and support activities for forestry", "113", "1153"],
    ["Information, culture and recreation [51,71]", "51", "71"],
    ["Agriculture", "111-112", "1100", "1151-1152"]
]

def merge_rows_with_codes(codes: list[str], merge_name: str, df):
    # Filter to find rows to merge
    # print(f"check: {df.shape}")
    temp = "|".join(codes)
    r_codes = r"\((" + temp + ")\)"
    df_to_merge = df[df["NAICS"].str.contains(r_codes, regex = True)]

    # Group by year and sum the GDP values
    merged_values = df_to_merge.groupby("Year", as_index=False).agg({"VALUE": "sum"})

    merged_values["NAICS"] = merge_name
    merged_values["Province"] = "Alberta"
    merged_values["Value"] = "Current dollars"
    # print(merged_values)

    # 3. Remove the individual industry rows
    df = df[~df["NAICS"].str.contains(r_codes, regex = True)]

    # 4. Append the new merged row
    df = pd.concat([df, merged_values], ignore_index=True)

    # print(result[result["NAICS"] == merge_name])

    # Sort the result by year for better readability
    df.sort_values(by=["Year", "NAICS"], inplace=True)

    return df

for item in classes_to_merge:
    gdp_df_cleaned = merge_rows_with_codes(item[1:], item[0], gdp_df_cleaned)

# Drop rows with NAICS values that contain letters
pattern = r"\((?=.*[a-zA-Z])[a-zA-Z0-9, \-\\s]*\)"
# print(f"sub : {gdp_df_cleaned.shape}")
gdp_df_cleaned = gdp_df_cleaned[~gdp_df_cleaned["NAICS"].str.contains(pattern, regex = True)]
# print(gdp_df_cleaned.shape)

# Drop remaining sub classes
pattern = r"\((\d{3},){0,3}(\d{3},)?(\d{3},)?(\d{3},)?(\d{3},)?\)"
# print(f"remaining: {gdp_df_cleaned.shape}")
gdp_df_cleaned = gdp_df_cleaned[~gdp_df_cleaned["NAICS"].str.contains(pattern, regex = True)]
# print(gdp_df_cleaned.shape)

# check if gdp values exist for each year by class
```

```
gdp_df_cleaned = gdp_df_cleaned[["NAICS", "VALUE", "Year"]].pivot(index="NAICS",
#gdp_df_cleaned.head()
```

/tmp/ipykernel_1506/1075602296.py:15: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.e
xtract.

```
df_to_merge = df[df["NAICS"].str.contains(r_codes, regex = True)]
```

/tmp/ipykernel_1506/1075602296.py:26: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.e
xtract.

```
df = df[~df["NAICS"].str.contains(r_codes, regex = True)]
```

/tmp/ipykernel_1506/1075602296.py:15: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.e
xtract.

```
df_to_merge = df[df["NAICS"].str.contains(r_codes, regex = True)]
```

/tmp/ipykernel_1506/1075602296.py:26: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.e
xtract.

```
df = df[~df["NAICS"].str.contains(r_codes, regex = True)]
```

/tmp/ipykernel_1506/1075602296.py:15: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.e
xtract.

```
df_to_merge = df[df["NAICS"].str.contains(r_codes, regex = True)]
```

/tmp/ipykernel_1506/1075602296.py:26: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.e
xtract.

```
df = df[~df["NAICS"].str.contains(r_codes, regex = True)]
```

/tmp/ipykernel_1506/1075602296.py:50: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.e
xtract.

```
gdp_df_cleaned = gdp_df_cleaned[~gdp_df_cleaned['NAICS'].str.contains(pattern,
regex=True)]
```

```
In [51]: # fill in missing values with median
gdp_df_cleaned.iloc[:, 1:] = gdp_df_cleaned.iloc[:, 1:].apply(lambda row: row.fi

gdp_df_cleaned = gdp_df_cleaned.melt(id_vars="NAICS", var_name="Year", value_nam

gdp_df_cleaned.sort_values(by=["NAICS", "Year"], inplace=True)
gdp_df_cleaned["Province"] = "Alberta"
gdp_df_cleaned["Value"] = "Current dollars"
```

```
In [52]: display(gdp_df_cleaned.isna().sum())
display(gdp_df_cleaned["NAICS"].unique())
#dropping Value column
gdp_df_cleaned = gdp_df_cleaned.drop('Value', axis=1)
```

```
NAICS      0
Year       0
VALUE      0
Province   0
Value      0
dtype: int64
```

```
array(['Accommodation and food services [72]',
      'Administrative and support, waste management and remediation services [56]',
      'Agriculture, forestry, fishing and hunting [11]',
      'Construction [23]', 'Educational services [61]',
      'Finance and insurance [52]',
      'Forestry and logging and support activities for forestry',
      'Health care and social assistance [62]',
      'Information, culture and recreation [51,71]',
      'Management of companies and enterprises [55]',
      'Manufacturing [31-33]',
      'Mining, quarrying, and oil and gas extraction [21]',
      'Other services (except public administration) [81]',
      'Professional, scientific and technical services [54]',
      'Public administration [91]',
      'Real estate and rental and leasing [53]', 'Retail trade [44-45]',
      'Transportation and warehousing [48-49]', 'Utilities [22]',
      'Wholesale trade [41]'], dtype=object)
```

Dataframe to SQL - GDP

```
In [53]: gdp_df_cleaned.to_sql(name='GDP_Data', if_exists='replace', con=engine, index
```

```
Out[53]: 480
```

```
In [54]: #Cleaning Industry column by replacing "[" to "]" with "" using REGEXP_REPLACE
industry_cleaning_query = """UPDATE GDP_Data
SET NAICS = REGEXP_REPLACE(NAICS, '\\\\[.?!\\\\]', '')
WHERE NAICS REGEXP '\\\\[.?!\\\\]';"""
conn.execute(text(industry_cleaning_query))
conn.commit()
```

```
In [55]: #Renaming column names
rename_queries = ["ALTER TABLE GDP_Data RENAME COLUMN `NAICS` TO `Industry`;",
                  "ALTER TABLE GDP_Data RENAME COLUMN `VALUE` TO `GDP_Value_mill`"]

for query in rename_queries:
    conn.execute(text(query))
conn.commit()
```

```
In [56]: #Aggregation of

# 1. "Information and cultural industries" and "Arts, entertainment and recreation"
# 2. "Management of companies and enterprises" and "Administrative and support, waste management and remediation services"

#Change names of Information and cultural industries and Arts, entertainment and recreation
# Management of companies and enterprises and Administrative and support, waste management and remediation services

renaming_industry_categories_query = """UPDATE GDP_Data
SET Industry =
CASE
WHEN Industry = "Retail trade" THEN "Wholesale trade"
WHEN Industry = "Wholesale trade" THEN "Wholesale trade"
WHEN Industry = "Management of companies and enterprises" THEN "Management of companies and enterprises"
WHEN Industry = "Administrative and support, waste management and remediation services" THEN "Administrative and support, waste management and remediation services"
END
WHERE Industry IN ("Retail trade",
                  "Wholesale trade",
                  "Management of companies and enterprises",
                  "Administrative and support, waste management and remediation services")
```

"Administrative and support, waste manage

```
conn.execute(text(renaming_industry_categories_query))
conn.commit()

# Display the new table to confirm the names are changed
new_industries_data = pd.read_sql("SELECT DISTINCT Industry FROM GDP_Data;",engi
display(new_industries_data)
```

	Industry
0	Accommodation and food services
1	Business, building and other support services
2	Agriculture, forestry, fishing and hunting
3	Construction
4	Educational services
5	Finance and insurance
6	Forestry and logging and support activities fo...
7	Health care and social assistance
8	Information, culture and recreation
9	Manufacturing
10	Mining, quarrying, and oil and gas extraction
11	Other services (except public administration)
12	Professional, scientific and technical services
13	Public administration
14	Real estate and rental and leasing
15	Wholesale and retail trade
16	Transportation and warehousing
17	Utilities

```
In [57]: # Creating final table
gdp_final_table_create_query =[
    "DROP TABLE IF EXISTS Final_GDP_Data;",

    """CREATE TABLE Final_GDP_Data AS
    SELECT
    YEAR,
    Province,
    Industry,
    CASE
    WHEN Industry IN (
    'Business, building and other support services',
    'Wholesale and retail trade')
    THEN SUM(COALESCE(CAST(GDP_Value_million AS FLOAT), 0))
    ELSE COALESCE(CAST(GDP_Value_million AS FLOAT), 0)
    END AS GDP_Value_million
```



```

        FROM
        GDP_Data
        GROUP BY
        YEAR,
        Province,
        Industry;"""]

for query in gdp_final_table_create_query:
    conn.execute(text(query))
conn.commit()
check_table_query = "SELECT * FROM Final_GDP_Data;"
table_data = pd.read_sql(check_table_query,engine)
display(table_data.head())

```

	YEAR	Province	Industry	GDP_Value_million
0	2000	Alberta	Accommodation and food services	3335.899902
1	2000	Alberta	Agriculture, forestry, fishing and hunting	2314.699951
2	2000	Alberta	Business, building and other support services	3690.300049
3	2000	Alberta	Construction	9407.599609
4	2000	Alberta	Educational services	5099.399902

In [58]: `table_data.shape`

Out[58]: (432, 4)

In [59]: *#Converting gdp table to CSV*
`table_data.to_csv("gdpfinal.csv",index=False)`

Joining Tables

In [60]: *# Combining employment and avg weekly earnings*
`check_table_query1 = "SELECT * FROM Avg_week_earnings;"`
`table_data1 = pd.read_sql(check_table_query1,engine)`
`display(table_data1.shape)`
Checking final table
`check_table_query2 = "SELECT * FROM Employment_Count;"`
`table_data2 = pd.read_sql(check_table_query2,engine)`
`display(table_data2.shape)`

(4896, 6)

(5472, 6)

In [61]: *# Checking final table GDP*
`check_table_query3 = "SELECT * FROM Final_GDP_Data;"`
`table_data3 = pd.read_sql(check_table_query3,engine)`
`display(table_data3.shape)`

(432, 4)

In [62]: *# Joining Avg_week_earnings and Employment_Count - Left Join*
`join_query1 = [ "DROP TABLE IF EXISTS Earning_Employment;",`
`""""CREATE TABLE Earning_Employment`

```

        AS SELECT a.YEAR,a.MONTH,a.Industry,a.Avg_Weekly_Earnings,e.Emp1
        FROM Employment_Count e
        INNER JOIN
        Avg_week_earnings a
        ON a.Industry = e.Industry
        AND a.YEAR=e.YEAR AND a.MONTH=e.MONTH;"""]
for query in join_query1:
    conn.execute(text(query))
check_table_query4 = "SELECT * FROM Earning_Employment;"
table_data4 = pd.read_sql(check_table_query4,engine)
table_data4.shape

```

Out[62]: (4896, 5)

```

In [63]: # Joining Earning_Employment and Final_GDP_Data - Inner Join
join_query2 = """SELECT t.YEAR,t.MONTH,t.Industry,t.Avg_Weekly_Earnings,t.Employ
        FROM Earning_Employment t
        INNER JOIN
        Final_GDP_Data g
        ON t.Industry = g.Industry
        AND t.YEAR=g.YEAR;"""]

join_query2_data = pd.read_sql(join_query2, engine)
display(join_query2_data.head())
display(join_query2_data.shape)

```

	YEAR	MONTH	Industry	Avg_Weekly_Earnings	Employment	GDP_Value_millior
0	2001	1	Accommodation and food services	273.799988	119202.0	3441.300049
1	2001	1	Business, building and other support services	739.760010	65531.0	4414.899902
2	2001	1	Construction	864.200012	80249.0	10361.000000
3	2001	1	Educational services	638.469971	100098.0	5574.899902
4	2001	1	Finance and insurance	0.000000	0.0	5872.600098

(4692, 6)

```

In [64]: #Converting final table to CSV file
join_query2_data.to_csv("finalDataset.csv",index=False)

```

```

In [65]: #Final joined table creation
joined_dataset=pd.read_csv("finalDataset.csv")
final_df = joined_dataset.to_sql('main_table', engine,if_exists='replace', index

```

```

In [66]: # Create a temporary table with industry averages
createtable = """CREATE TEMPORARY TABLE temp_industry_avgs AS
SELECT Industry,AVG(Avg_Weekly_Earnings) AS industry_avg

```

```
FROM main_table WHERE Avg_Weekly_Earnings > 0 GROUP BY Industry;"""
conn.execute(text(createtemptable))
```

Out[66]: <sqlalchemy.engine.cursor.CursorResult at 0x73d21d2bcb80>

```
In [67]: #Missing Avg_Weekly_Earnings filled with industry averages in a new table.
createfinal = [
    "DROP TABLE IF EXISTS finalnewDataset_filled;",
    """CREATE TABLE finalnewDataset_filled AS
SELECT
    f.YEAR,
    f.MONTH,
    f.Industry,
    f.Employment,
    f.GDP_Value_million,
    COALESCE(NULLIF(f.Avg_Weekly_Earnings, 0), t.industry_avg) AS Avg_Weekly_Ear
FROM main_table f
LEFT JOIN temp_industry_avgs t ON f.Industry = t.Industry;"""]
for query in createfinal:
    conn.execute(text(query))
```

```
In [68]: avg_filed_data = pd.read_sql_query("SELECT * FROM finalnewDataset_filled ;",engi
display(avg_filed_data.head())
```

	YEAR	MONTH	Industry	Employment	GDP_Value_million	Avg_Weekly_Earnings
0	2001	1	Accommodation and food services	119202.0	3441.300049	273.7
1	2001	2	Accommodation and food services	119966.0	3441.300049	270.8
2	2001	3	Accommodation and food services	121648.0	3441.300049	253.1
3	2001	4	Accommodation and food services	123114.0	3441.300049	264.7
4	2001	5	Accommodation and food services	124528.0	3441.300049	250.0



Guiding Questions - Part 2

6. Which are the top 5 industries with the highest and lowest average weekly earnings?

```
In [69]: # Highest and Lowest Paying Industries :
top5_avg_pay = """
(SELECT
    'Highest Paying' AS Category,
    Industry,
```

```

        ROUND(AVG(Avg_Weekly_Earnings_filled), 2) AS Avg_Weekly_Pay
FROM finalnewDataset_filled
WHERE Avg_Weekly_Earnings_filled > 0
GROUP BY Industry
ORDER BY Avg_Weekly_Pay DESC
LIMIT 5)

UNION ALL

(SELECT
    'Lowest Paying' AS Category,
    Industry,
    ROUND(AVG(Avg_Weekly_Earnings_filled), 2) AS Avg_Weekly_Pay
FROM finalnewDataset_filled
WHERE Avg_Weekly_Earnings_filled > 0
GROUP BY Industry
ORDER BY Avg_Weekly_Pay ASC
LIMIT 5)

ORDER BY Category DESC, Avg_Weekly_Pay DESC;
"""
top5_avg_pay = pd.read_sql_query(top5_avg_pay, engine)
display(top5_avg_pay)

```

	Category	Industry	Avg_Weekly_Pay
0	Lowest Paying	Health care and social assistance	840.39
1	Lowest Paying	Other services (except public administration)	816.46
2	Lowest Paying	Information, culture and recreation	812.40
3	Lowest Paying	Wholesale and retail trade	747.31
4	Lowest Paying	Accommodation and food services	393.86
5	Highest Paying	Mining, quarrying, and oil and gas extraction	1890.51
6	Highest Paying	Utilities	1696.77
7	Highest Paying	Professional, scientific and technical services	1331.88
8	Highest Paying	Construction	1289.67
9	Highest Paying	Finance and insurance	1225.47

```

In [70]: industry_avg = joined_dataset.groupby('Industry')['Avg_Weekly_Earnings'].mean().
top_5_highest = industry_avg.head(5)
top_5_lowest = industry_avg.tail(5)
plt.figure(figsize=(12, 6))

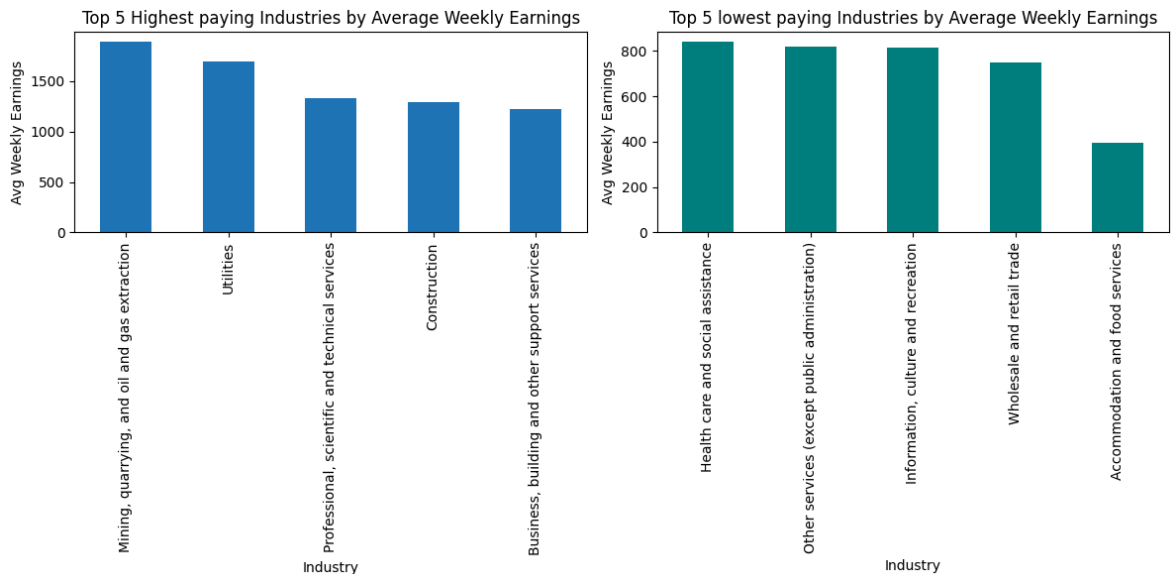
# Graph for Top 5 highest industries
plt.subplot(1, 2, 1)
top_5_highest.plot(kind='bar')
plt.title('Top 5 Highest paying Industries by Average Weekly Earnings')
plt.xticks()
plt.ylabel('Avg Weekly Earnings')

# Graph for Top 5 Lowest industries
plt.subplot(1, 2, 2)
top_5_lowest.plot(kind='bar', color='teal')

```

```
plt.title('Top 5 lowest paying Industries by Average Weekly Earnings')
plt.xticks()
plt.ylabel('Avg Weekly Earnings')

plt.tight_layout()
plt.show()
```



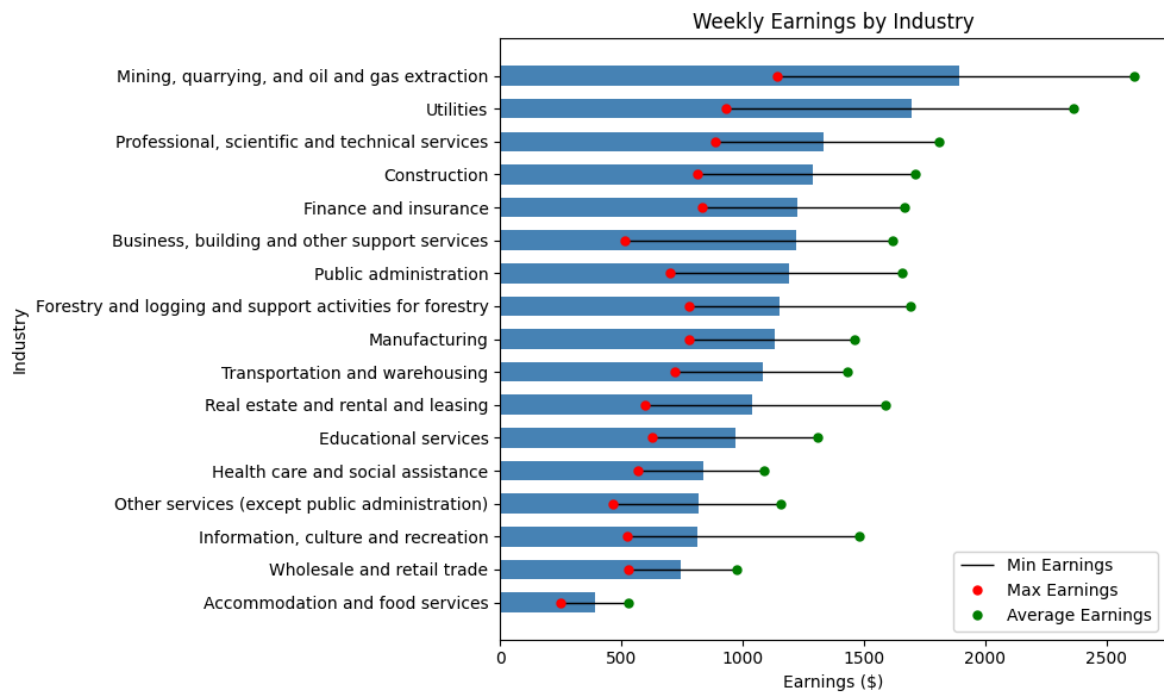
The graph compares the top 5 highest-paying and top 5 lowest-paying industries based on average weekly earnings. It highlights that industries like Mining, Utilities, and Professional Services offer the highest wages, whereas Accommodation, Retail, and Social Assistance sectors have the lowest earnings.

7. What is the overall distribution of average weekly earnings across different industries?

```
In [71]: #Overall Earnings Distribution by Industry
overall_earning = """SELECT
    Industry,
    MIN(Avg_Weekly_Earnings_filled) AS Min_Earnings,
    MAX(Avg_Weekly_Earnings_filled) AS Max_Earnings,
    AVG(Avg_Weekly_Earnings_filled) AS Avg_Earnings,
    COUNT(*) AS total_number
FROM (
    SELECT DISTINCT Industry, Avg_Weekly_Earnings_filled
    FROM finalnewDataset_filled
    WHERE Avg_Weekly_Earnings_filled IS NOT NULL
) AS distinct_data
GROUP BY Industry
ORDER BY Avg_Earnings DESC;"""
overall_earning_data = pd.read_sql_query(overall_earning, engine)
display(overall_earning_data)
```

	Industry	Min_Earnings	Max_Earnings	Avg_Earnings	total_number
0	Mining, quarrying, and oil and gas extraction	1143.900024	2615.479980	1890.508626	276
1	Utilities	929.119995	2361.270020	1696.774634	276
2	Professional, scientific and technical services	887.090027	1806.989990	1331.883004	276
3	Construction	815.440002	1709.420044	1289.672463	276
4	Finance and insurance	834.440002	1668.329956	1225.472692	265
5	Business, building and other support services	515.289978	1619.425018	1219.001540	276
6	Public administration	700.960022	1659.079956	1190.684998	276
7	Forestry and logging and support activities fo...	780.469971	1691.969971	1151.094765	255
8	Manufacturing	778.090027	1463.500000	1130.818803	276
9	Transportation and warehousing	720.219971	1431.689941	1084.067756	276
10	Real estate and rental and leasing	598.000000	1589.800049	1040.903826	265
11	Educational services	626.349976	1308.709961	972.919785	276
12	Health care and social assistance	568.299988	1086.329956	839.931346	275
13	Other services (except public administration)	464.570007	1156.880005	816.455401	276
14	Information, culture and recreation	524.940018	1481.090027	811.454583	275
15	Wholesale and retail trade	528.359985	976.150024	747.312535	276
16	Accommodation and food services	249.270004	528.429993	393.715949	274

```
In [72]: overall_earning_data_visual= overall_earning_data.sort_values('Avg_Earnings')
plt.figure(figsize=(10, 6))
plt.barh(overall_earning_data_visual['Industry'], overall_earning_data_visual['A
for i, industry in enumerate(overall_earning_data_visual['Industry']):
    min_val = overall_earning_data_visual['Min_Earnings'].iloc[i]
    max_val = overall_earning_data_visual['Max_Earnings'].iloc[i]
    plt.hlines(y=i, xmin=min_val, xmax=max_val, color='black', linewidth=1)
    plt.plot(min_val, i, 'ro', markersize=5)
    plt.plot(max_val, i, 'go', markersize=5)
plt.title('Weekly Earnings by Industry')
plt.xlabel('Earnings ($)')
plt.ylabel('Industry')
plt.legend(['Min Earnings', 'Max Earnings', 'Average Earnings'])
plt.tight_layout()
plt.show()
```



This graph shows the range of weekly earnings across different industries. The blue bars represent the average earnings, the red dots show the highest earnings, and the green dots show the average earnings. Industries like mining and utilities have higher average and maximum earnings, while industries like accommodation and food services have the lowest. This helps us understand how pay varies across industries and which sectors offer better income opportunities.

```
In [73]: # Monthly Earnings Trends by Industry
trends="""SELECT
    Industry,
    MONTH,
    AVG(Avg_Weekly_Earnings_filled) AS Avg_Weekly_Earnings
FROM finalnewDataset_filled
WHERE Avg_Weekly_Earnings_filled > 0
GROUP BY Industry, MONTH
ORDER BY Industry, MONTH;"""
trends_data = pd.read_sql_query(trends, engine)
display(trends_data.head())
```

	Industry	MONTH	Avg_Weekly_Earnings
0	Accommodation and food services	1	382.536520
1	Accommodation and food services	2	388.129563
2	Accommodation and food services	3	384.457389
3	Accommodation and food services	4	390.918258
4	Accommodation and food services	5	392.192608

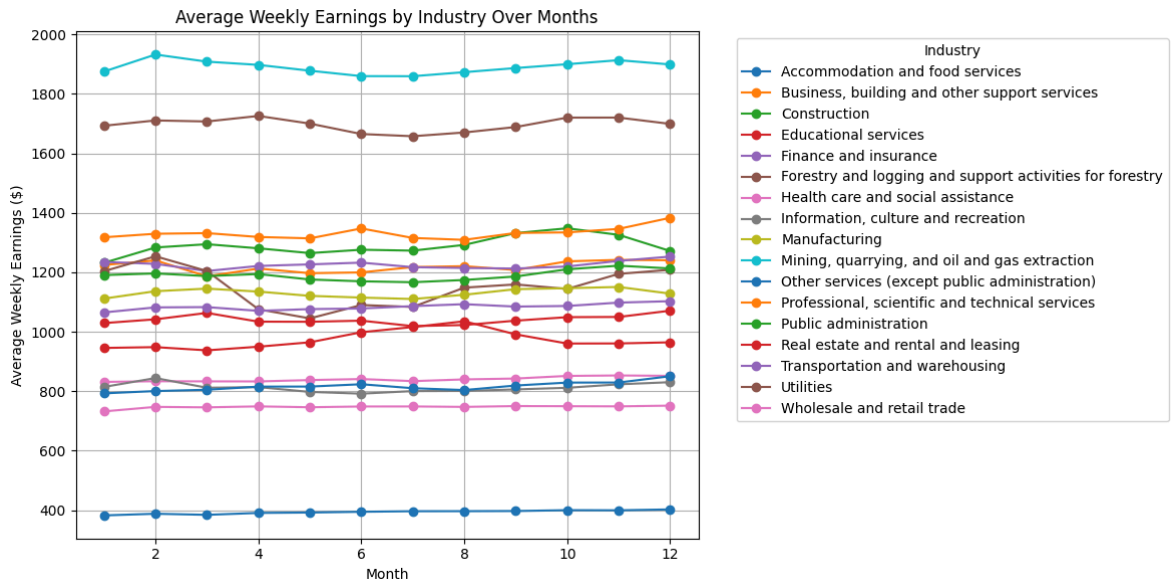
```
In [74]: plt.figure(figsize=(12, 6))
```

```

for industry in trends_data['Industry'].unique():
    industry_data = trends_data[trends_data['Industry'] == industry]
    plt.plot(industry_data['MONTH'], industry_data['Avg_Weekly_Earnings'],
             label=industry, marker='o')

plt.xlabel('Month')
plt.ylabel('Average Weekly Earnings ($)')
plt.title('Average Weekly Earnings by Industry Over Months')
plt.legend(title='Industry', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid()
plt.tight_layout()
plt.show()

```



The line chart shows that high-paying industries like "Mining, quarrying, and oil and gas extraction" and "Utilities" maintain consistently higher average weekly earnings across the months, while industries such as "Accommodation and food services" and "Wholesale and retail trade" remain at the lower end.

8. What are the average weekly earnings and average employment for each industry?

```

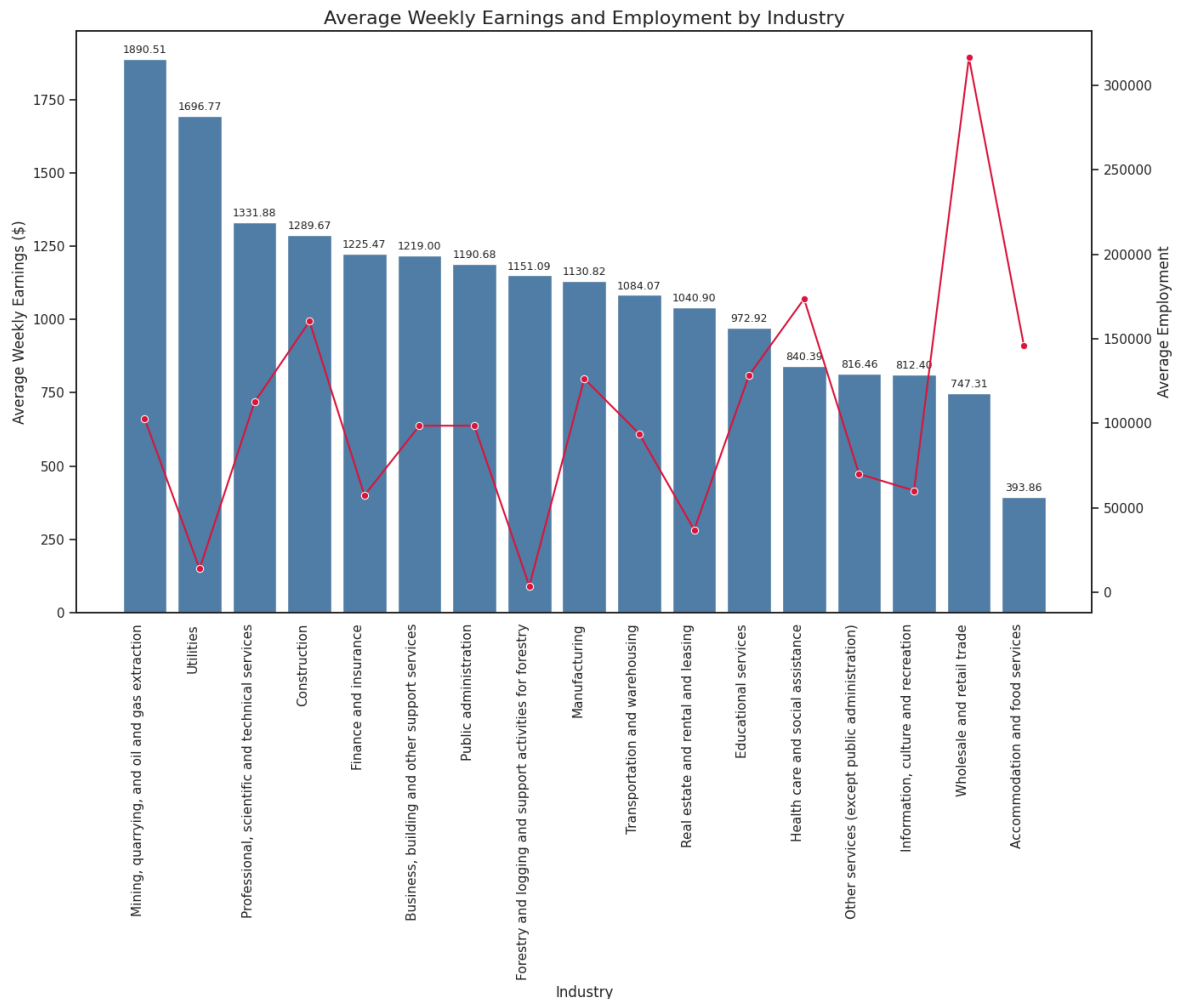
In [75]: avgEarning_avgEmploy="""SELECT
        Industry,
        AVG(Avg_Weekly_Earnings) AS Avg_Wage,
        AVG(Employment) AS Avg_Employment,
        COUNT(*) AS Data_Points
    FROM main_table
    WHERE Avg_Weekly_Earnings > 0 AND Employment > 0
    GROUP BY Industry
    ORDER BY Avg_Wage DESC;""

avgEarning_avgEmploy_data = pd.read_sql_query(avgEarning_avgEmploy, engine)
display(avgEarning_avgEmploy_data)

```


	Industry	Avg_Wage	Avg_Employment	Data_Points
0	Mining, quarrying, and oil and gas extraction	1890.508626	102709.905797	276
1	Utilities	1696.774634	13991.971014	276
2	Professional, scientific and technical services	1331.883004	112591.028986	276
3	Construction	1289.672463	160314.155797	276
4	Finance and insurance	1225.472692	57442.564394	264
5	Business, building and other support services	1219.001540	98578.728261	276
6	Public administration	1190.684998	98620.847826	276
7	Forestry and logging and support activities fo...	1151.094765	3842.771654	254
8	Manufacturing	1130.818803	126386.967391	276
9	Transportation and warehousing	1084.067756	93843.083333	276
10	Real estate and rental and leasing	1040.903826	37041.818182	264
11	Educational services	972.919785	128538.905797	276
12	Health care and social assistance	840.391595	173609.126812	276
13	Other services (except public administration)	816.455401	70105.025362	276
14	Information, culture and recreation	812.404041	60201.750000	276
15	Wholesale and retail trade	747.312535	316292.550725	276
16	Accommodation and food services	393.858514	146084.496377	276

```
In [76]: #Plotting Average weekly earning vs Average Employment by industry.
sns.set(style="white")
avgEarning_avgEmploy_data = avgEarning_avgEmploy_data.sort_values(by='Avg_Wage',
fig, ax1 = plt.subplots(figsize=(14, 12))
sns.barplot(data=avgEarning_avgEmploy_data, x='Industry', y='Avg_Wage', color='s
plt.xticks(rotation=90, ha='right')
ax2 = ax1.twinx()
sns.lineplot(data=avgEarning_avgEmploy_data, x='Industry', y='Avg_Employment', m
for i, row in avgEarning_avgEmploy_data.iterrows():
    ax1.text(i, row['Avg_Wage'] + 20, f"{row['Avg_Wage']:.2f}", ha='center', fon
ax1.set_ylabel('Average Weekly Earnings ($)', fontsize=12)
ax2.set_ylabel('Average Employment', fontsize=12)
ax1.set_xlabel('Industry', fontsize=12)
plt.title('Average Weekly Earnings and Employment by Industry', fontsize=16)
plt.tight_layout()
plt.show()
```



This graph shows the relationship between average weekly wages and average employment across different industries. The blue bars represent the average weekly wages for each industry, while the red line shows the average number of employees in those industries. Industries like mining and utilities have the highest average wages, but the number of employees is less. On the other hand, industries like wholesale and retail trade have more employees but lower average wages.

8.1. Do higher employment opportunities indicate lower wages? (Hypothesis testing)

```
In [77]: from scipy import stats

df = pd.read_sql_query("SELECT * FROM finalnewDataset_filled", engine)
df = df[(df['Avg_Weekly_Earnings_filled'] > 0) & (df['Employment'] > 0)]
# Calculate the 33rd and 66th percentiles for Employment
p33 = df['Employment'].quantile(0.33)
p66 = df['Employment'].quantile(0.66)
# Employment Level categories using the percentiles
def categorize_employment(emp):
    """Assigns 'Low', 'Medium', or 'High' based on employment percentiles."""
    if emp <= p33:
        return 'Low'
    elif emp <= p66:
```

```

        return 'Medium'
    else:
        return 'High'

df['Employment_Level'] = df['Employment'].apply(categorize_employment)

low = df[df['Employment_Level'] == 'Low']['Avg_Weekly_Earnings_filled']
medium = df[df['Employment_Level'] == 'Medium']['Avg_Weekly_Earnings_filled']
high = df[df['Employment_Level'] == 'High']['Avg_Weekly_Earnings_filled']
f_stat, p_value = stats.f_oneway(low, medium, high)
print(f"ANOVA Results: F-statistic = {f_stat:.2f}, p-value = {p_value:.4f}")
if p_value < 0.05:
    print("There is a significant difference between employment groups.")
else:
    print("No significant difference between employment groups.")

plt.figure(figsize=(8, 5))
sns.boxplot(x='Employment_Level', y='Avg_Weekly_Earnings_filled', hue='Employment_Level')
plt.title("Average Weekly Earnings by Employment Level")
plt.xlabel("Employment Level")
plt.ylabel("Avg Weekly Earnings")
plt.grid(True)
plt.show()

```

ANOVA Results: F-statistic = 191.48, p-value = 0.0000

There is a significant difference between employment groups.



- The ANOVA results show a high F-statistic of 191.48 and a p-value of 0.0000, indicating that there is a statistically significant difference in average weekly earnings between the different employment groups (Low, Medium, High). This suggests that employment levels have a clear impact on earnings.
- The chart illustrates that average weekly earnings are highest for the "Medium" employment level, ranging around 2,500 dollars, while the "High" and "Low"

employment levels show lower and more similar earnings, around 2,000 to 2,200 dollars, suggesting that moderate employment levels correlate with higher wages compared to extremes.

9. Which industries are biggest and least in terms of providing employment opportunities?

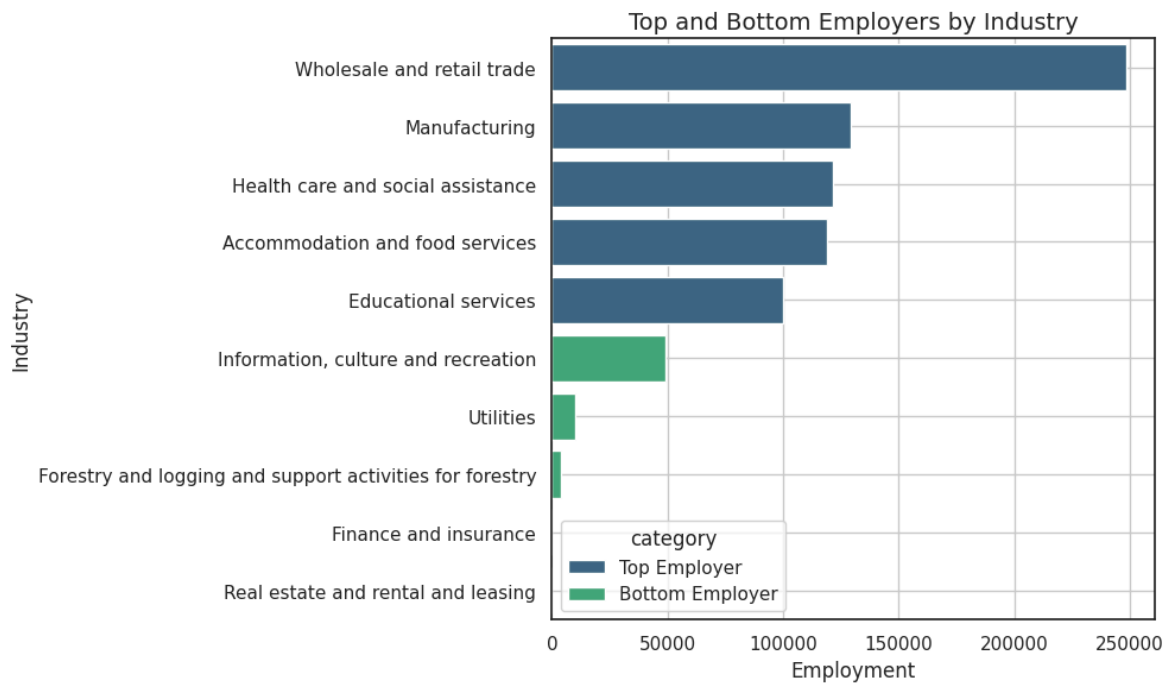
```
In [78]: employment_providers= """(SELECT 'Top Employer' AS category,
        Industry,Employment
        FROM main_table
        GROUP BY Industry
        ORDER BY Employment DESC LIMIT 5)
UNION ALL
(SELECT 'Bottom Employer' AS category,
        Industry,Employment
        FROM main_table
        GROUP BY Industry
        ORDER BY Employment ASC LIMIT 5)
ORDER BY category DESC,Employment DESC;"""

employment_providers_data = pd.read_sql_query(employment_providers, engine)
display(employment_providers_data)
```

	category	Industry	Employment
0	Top Employer	Wholesale and retail trade	248394.0
1	Top Employer	Manufacturing	129465.0
2	Top Employer	Health care and social assistance	121680.0
3	Top Employer	Accommodation and food services	119202.0
4	Top Employer	Educational services	100098.0
5	Bottom Employer	Information, culture and recreation	49291.0
6	Bottom Employer	Utilities	10140.0
7	Bottom Employer	Forestry and logging and support activities fo...	3735.0
8	Bottom Employer	Finance and insurance	0.0
9	Bottom Employer	Real estate and rental and leasing	0.0

```
In [79]: sns.set(style="white")
plt.figure(figsize=(10, 6))
sns.barplot(x='Employment', y='Industry', hue='category', data=employment_provid
plt.title('Top and Bottom Employers by Industry', fontsize=14)
plt.xlabel('Employment', fontsize=12)
plt.ylabel('Industry', fontsize=12)

plt.tight_layout()
plt.grid()
plt.show()
```



This bar chart shows the top and bottom employers by industry based on employment numbers. The Wholesale and Retail Trade industry is the largest employer, followed by Manufacturing and Health Care and Social Assistance. The Top Employer category is shown in dark orange, while the Bottom Employer category is in light orange. Industries such as Utilities and Forestry and Logging have the lowest employment numbers. The chart highlights the disparity in employment across different industries.

10. Which industries in Alberta lost the most jobs during the COVID period (March 2020 to December 2021)?

```
In [80]: covid_joblost="""
SELECT
    Industry,
    SUM(CASE WHEN Year < 2020 THEN Employment ELSE 0 END) AS Jobs_Before_COVID,
    SUM(CASE WHEN Year BETWEEN 2020 AND 2021 AND Month BETWEEN 3 AND 12 THEN Emp
    SUM(CASE WHEN Year < 2020 THEN Employment ELSE 0 END) -
    SUM(CASE WHEN Year BETWEEN 2020 AND 2021 AND Month BETWEEN 3 AND 12 THEN Emp
FROM finalnewDataset_filled
GROUP BY Industry
HAVING SUM(CASE WHEN Year < 2020 THEN Employment ELSE 0 END) > 0
ORDER BY Jobs_Lost DESC ; """
covid_joblost_data = pd.read_sql_query(covid_joblost, engine)
display(covid_joblost_data)
```

	Industry	Jobs_Before_COVID	Jobs_During_COVID	Jobs_Lost
0	Wholesale and retail trade	70996454.0	6589846.0	64406608.0
1	Construction	36055472.0	3217047.0	32838425.0
2	Health care and social assistance	36971378.0	4280650.0	32690728.0
3	Accommodation and food services	33454970.0	2524982.0	30929988.0
4	Manufacturing	29188264.0	2282588.0	26905676.0
5	Educational services	28528753.0	2733425.0	25795328.0
6	Professional, scientific and technical services	25157317.0	2301192.0	22856125.0
7	Mining, quarrying, and oil and gas extraction	23843101.0	1755543.0	22087558.0
8	Business, building and other support services	22134852.0	2079610.0	20055242.0
9	Public administration	22060061.0	2092896.0	19967165.0
10	Transportation and warehousing	20726271.0	2026891.0	18699380.0
11	Other services (except public administration)	16021548.0	1294860.0	14726688.0
12	Information, culture and recreation	13824717.0	1029998.0	12794719.0
13	Finance and insurance	11955721.0	1282555.0	10673166.0
14	Real estate and rental and leasing	7979449.0	734100.0	7245349.0
15	Utilities	3099758.0	293804.0	2805954.0
16	Forestry and logging and support activities fo...	876417.0	69343.0	807074.0

11. Did industries with higher GDP before COVID suffer less during the pandemic?

```
In [81]: covid_suffer = """
with yearly_gdp as (
    select
        Industry,
        YEAR,
        sum(distinct GDP_Value_million) / count(distinct MONTH) as annual_gdp
    from main_table
    where year between 2016 and 2022
    group by Industry, YEAR
),
```

```

yearly_growth as (
    select
        Industry,
        YEAR,
        annual_gdp,
        lag(annual_gdp) over (partition by Industry order by YEAR) as previous_y
    from yearly_gdp
)
select
    Industry,
    YEAR,
    ((annual_gdp - previous_year_gdp) / previous_year_gdp) * 100 as growth_rate
from yearly_growth
where previous_year_gdp is not NULL
order by Industry, YEAR;

"""
covid_suffer_data=pd.read_sql(covid_suffer, conn)
display(covid_suffer_data.head(10))

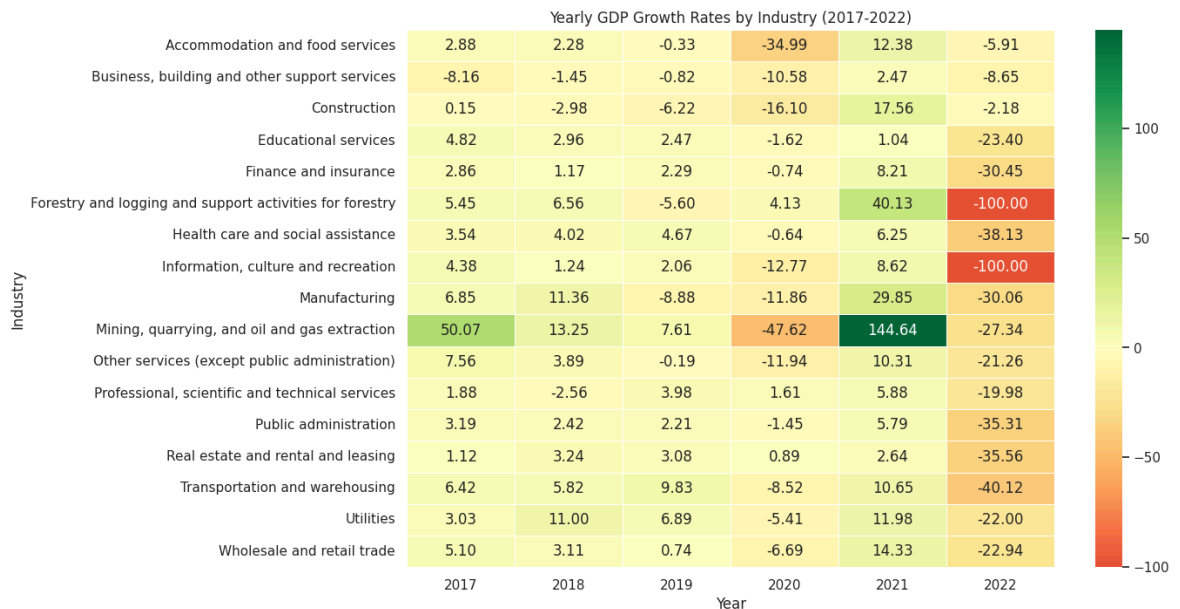
```

	Industry	YEAR	growth_rate
0	Accommodation and food services	2017	2.884856
1	Accommodation and food services	2018	2.282299
2	Accommodation and food services	2019	-0.325408
3	Accommodation and food services	2020	-34.991004
4	Accommodation and food services	2021	12.378549
5	Accommodation and food services	2022	-5.909713
6	Business, building and other support services	2017	-8.162649
7	Business, building and other support services	2018	-1.451338
8	Business, building and other support services	2019	-0.819446
9	Business, building and other support services	2020	-10.584735

```

In [82]: # Pivot the dataframe for heatmap format
pivot = covid_suffer_data.pivot(index = "Industry", columns = "YEAR", values = "
fig, ax = plt.subplots(figsize=(12, 8))
sns.heatmap(pivot, annot=True, fmt=".2f", cmap="RdYlGn", linewidths=0.5, center=
plt.title("Yearly GDP Growth Rates by Industry (2017-2022)")
plt.xlabel("Year")
plt.ylabel("Industry")
#plt.savefig('gdp_growth_2015_2020_heat.png', bbox_inches='tight')
plt.show()

```



```
In [83]: # Pre-COVID average GDP data
q1 = """
create or replace temporary table pre_covid_gdp as
select
    Industry,
    avg(distinct GDP_Value_million) as avg_gdp_pre_covid
from main_table
where YEAR between 2015 and 2018
group by Industry;
"""

# calculate GDP change over COVID period
q2 = """
create or replace temporary table covid_gdp as
select
    Industry,
    avg(distinct GDP_Value_million) as avg_gdp_covid
from main_table
where YEAR between 2019 and 2021
group by Industry;
"""

# Table aggregation
query = """
select
    p.Industry,
    p.avg_gdp_pre_covid,
    c.avg_gdp_covid,
    round(((c.avg_gdp_covid - p.avg_gdp_pre_covid) / p.avg_gdp_pre_covid) * 100,
from pre_covid_gdp p
join covid_gdp c on p.Industry = c.Industry
order by gdp_change_percent ASC;
"""

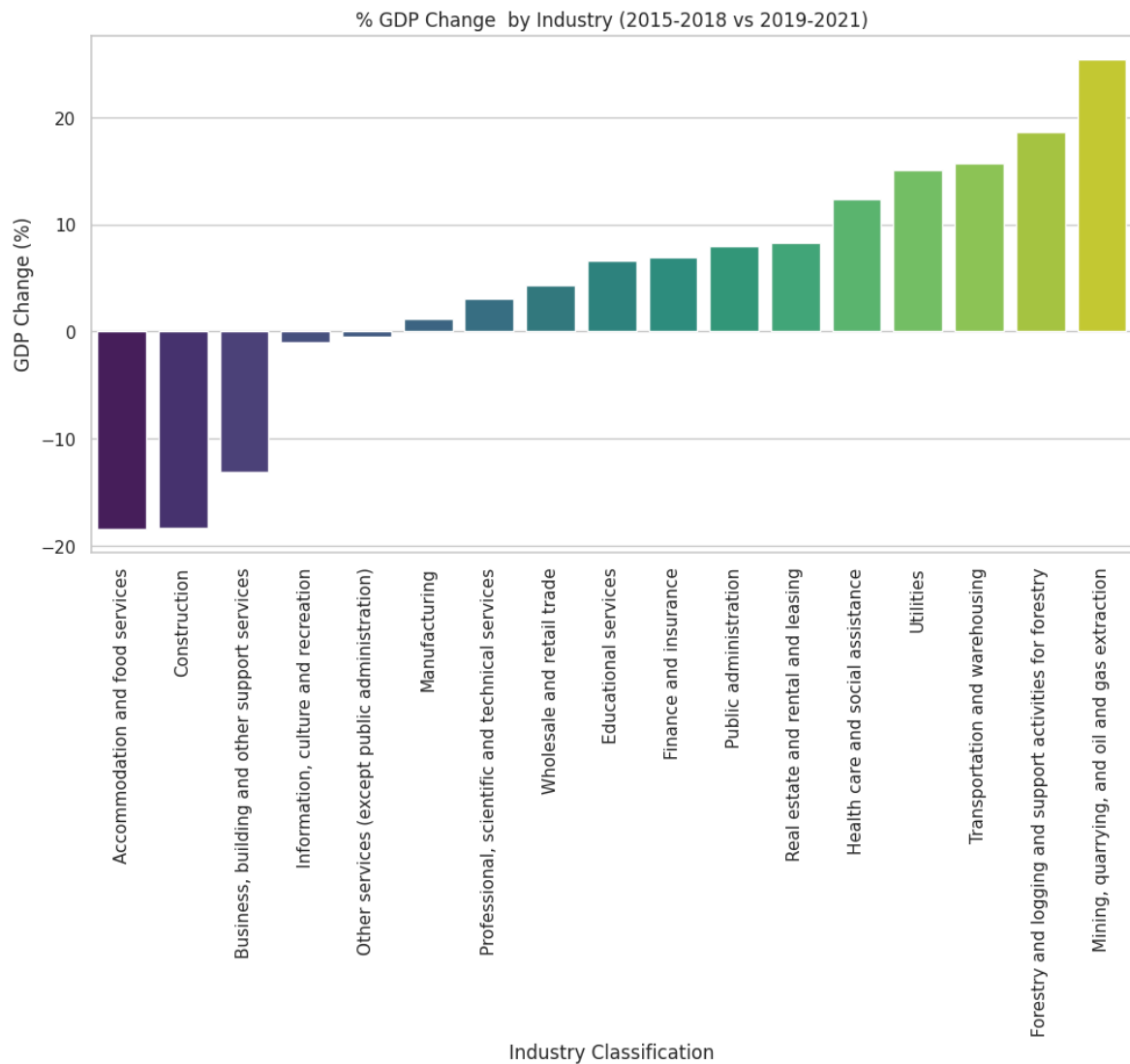
for q in [q1, q2]:
    conn.execute(sq.text(q))
df = pd.read_sql(query, conn)
display(df.head())

# Plot
sns.set_theme(style="whitegrid")
df_sorted = df.sort_values('gdp_change_percent', ascending=True)
```



```
fig, ax = plt.subplots(figsize=(12, 6))
sns.barplot(data=df_sorted, x='Industry', y='gdp_change_percent', ax=ax, hue='In
ax.set_title('% GDP Change by Industry (2015-2018 vs 2019-2021)')
ax.set_xlabel('Industry Classification')
ax.set_ylabel('GDP Change (%)')
plt.xticks(rotation=90)
plt.show()
```

	Industry	avg_gdp_pre_covid	avg_gdp_covid	gdp_change_percent
0	Accommodation and food services	7298.025024	5955.200033	-18.40
1	Construction	31816.725098	25993.866536	-18.30
2	Business, building and other support services	10295.275269	8945.966848	-13.11
3	Information, culture and recreation	8977.175049	8889.266602	-0.98
4	Other services (except public administration)	6641.424927	6608.399902	-0.50



- The Mining, Quarrying, and Oil & Gas Extraction industry maintained the #1 rank in GDP across all years except for in 2020**, when it fell to rank 2. The drop in 2020 could have been due to the COVID-19 pandemic or market fluctuations in oil and gas. It recovered in 2021, with its GDP reaching its highest recorded value \$88.8 billion, then stabilized in 2022 at \$64.5 billion.
- The Real Estate and Rental and Leasing Stability industry consistently remained at rank 2 from 2017 to 2021. However, in 2022, it dropped to rank 3, showing a significant decline in GDP from \$40.7 billion dollars in 2021, to \$26.2 billion in 2022. The decline suggests either a slowdown in real estate activity or reduced demand in rental and leasing markets.
- Construction showed a sharp drop in 2021 but recovered in 2022. The construction industry maintained rank 3 until 2020 but dropped to rank 5 in 2021. It however rebound in 2022, climbing back to rank 2 with GDP \$26.6 billion. The decline in its growth could be explained by restrictions enforced during the pandemic which is why the trend reverses after economic activities resumed. Out of the 5 highest producing industries pre-covid, Construction seemed to be the most negatively impacted.
- Wholesale and Retail Trade ranked 4th from 2017 to 2019, but moved up to rank 3 in 2020 and 2021. This trend could be driven by pandemic-driven shifts in retail, e-commerce, and supply chain activity. Post-pandemic, it falls back to rank 4 in 2022, suggesting a return to more typical trade patterns. Retail trade and wholesale seemed to benefit from the pandemic, likely due to pandemic-related changes in consumer behavior.
- Manufacturing ranked 5th every year except 2021, when it moves up to rank 4 but only temporarily. More broadly, Manufacturing seems to show a steady decline in GDP declining from \$25.2 billion in 2017, to \$20.5 billion in 2022. This is likely due to employment disruptions caused by periodic lockdowns during the pandemic or changes in consumer consumption patterns of manufactured foods due to post-pandemic inflation.

12. Which industries have seen the highest growth in GDP, and how do they compare in employment levels/average earnings?

```
In [84]: # Get yearly GDP values
q1 = """
create or replace temporary table yearly_gdp as
select
    Industry,
    YEAR,
    sum(distinct GDP_Value_million) as yearly_gdp
from main_table
group by Industry, YEAR;
"""

# GDP growth
```

```

q2 = """

create or replace temporary table gdp_growth as
select
    Industry,
    avg(case when YEAR between 2001 and 2005 then yearly_gdp else NULL end) as g
    avg(case when YEAR between 2019 and 2023 then yearly_gdp else NULL end) as g
from yearly_gdp
group by Industry;
"""

# Employment and earnings trends
q3 = """
create or replace temporary table employment_earnings as
select
    Industry,
    avg(case when YEAR between 2001 and 2005 then Employment else NULL end) as e
    avg(case when YEAR between 2019 and 2023 then Employment else NULL end) as e
    avg(case when YEAR between 2001 and 2005 then Avg_Weekly_Earnings else NULL
    avg(case when YEAR between 2019 and 2023 then Avg_Weekly_Earnings else NULL
from main_table
group by Industry;
"""

# Join and compare
query = """
select
    g.Industry,
    round(((g.gdp_recent - g.gdp_early) / g.gdp_early) * 100, 2) as gdp_growth_p
    round(((e.emp_recent - e.emp_early) / e.emp_early) * 100, 2) as employment_g
    round(((e.earnings_recent - e.earnings_early) / e.earnings_early) * 100, 2)
from gdp_growth g
join employment_earnings e on g.Industry = e.Industry
order by gdp_growth_percent desc;
"""

for q in [q1, q2, q3]:
    conn.execute(sq.text(q))

df = pd.read_sql(query, conn)
display(df.head())

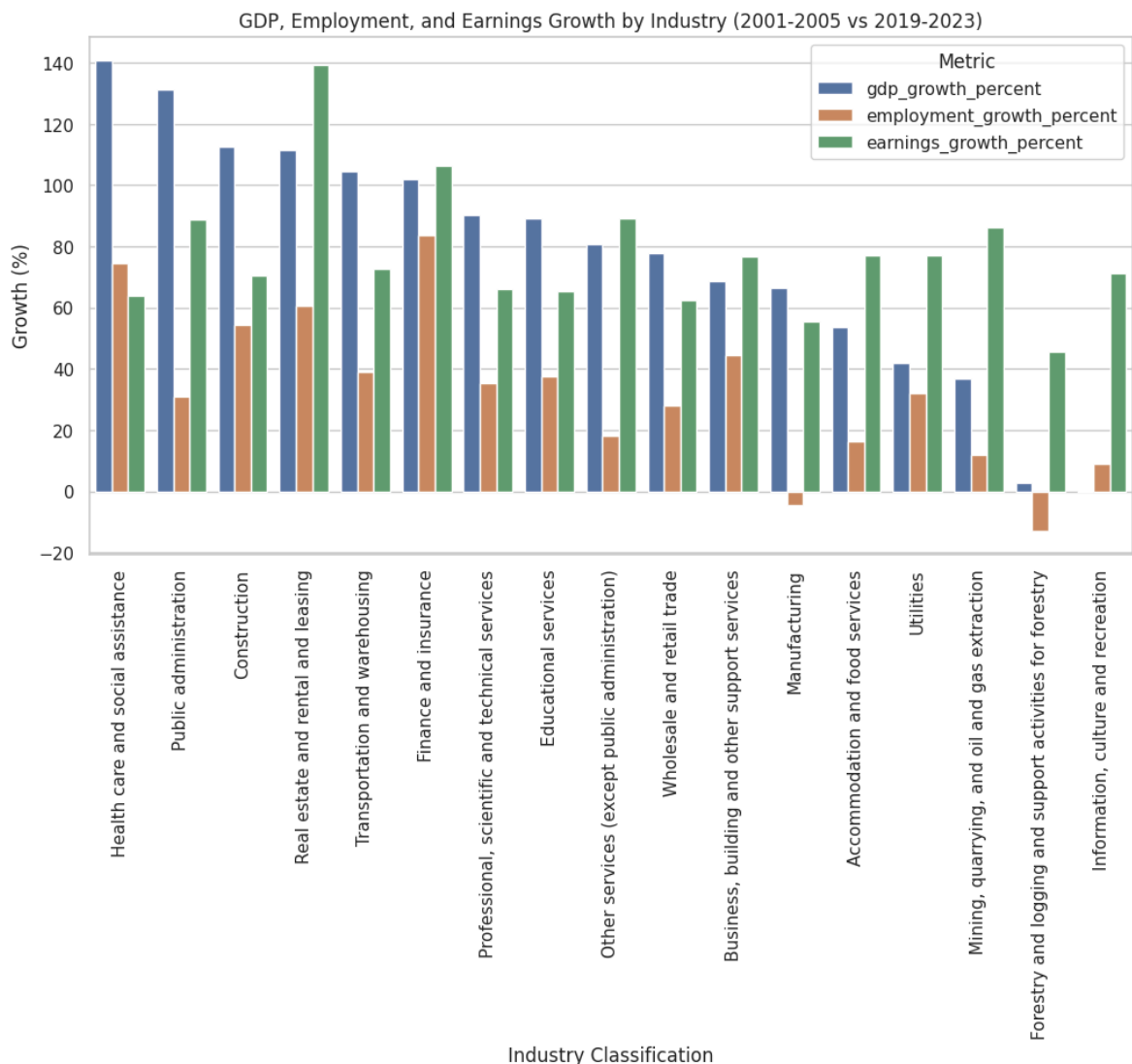
```

	Industry	gdp_growth_percent	employment_growth_percent	earnings_growth_perc
0	Health care and social assistance	140.97	74.51	63
1	Public administration	131.26	30.91	88
2	Construction	112.48	54.26	70
3	Real estate and rental and leasing	111.67	60.61	139
4	Transportation and warehousing	104.54	39.12	72

```
In [85]: # Plot heatmap of correlation matrix
corr = df[['gdp_growth_percent', 'employment_growth_percent', 'earnings_growth_p
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap='GnBu', center=0, ax=ax)
plt.title('Correlation Matrix: GDP, Employment, and Earnings Growth')
plt.show()
```



```
In [86]: sns.set(style="whitegrid")
df_melted = df.melt(id_vars='Industry',
                    value_vars=['gdp_growth_percent', 'employment_growth_per
                                cent', 'earnings_growth_percent'],
                    var_name='Metric', value_name='Growth (%)')
fig, ax = plt.subplots(figsize=(12, 6))
sns.barplot(data=df_melted, x='Industry', y='Growth (%)', hue='Metric', ax=ax)
ax.set_title('GDP, Employment, and Earnings Growth by Industry (2001-2005 vs 2019-2023)')
ax.set_xlabel('Industry Classification')
ax.set_ylabel('Growth (%)')
plt.xticks(rotation=90)
plt.legend(title='Metric')
plt.show()
```



1. Volatility in the Finance and Insurance Sector

- The finance and insurance industry shows high fluctuations, with strong positive growth from 2005 to 2006 but sharply declines in 2021 (-2.54%) and 2023 (0.00%).
- The construction industry exhibits high growth in the mid-2000s, peaking around 2006 to 2007. However, it experienced major declines in 2009 (-1.66%) and 2016 (-1.55%), potentially due to economic crises like the 2008 market crisis and the 2016

Alberta Oil crisis. A strong rebound in 2021 (1.46%) could indicate a post-pandemic recovery in housing or infrastructure projects.

- Accommodation and Food Services dropped sharply in 2020 (-2.92%), likely due to COVID-19 lockdowns, travel restrictions, and reduced consumer activity. The industry made a strong recovery in 2021 (1.03%), which suggests that restaurants, hotels, and tourism were able to recover relatively quickly post-pandemic. The decline from 2022 to 2023 might be due to economic instability and inflation which have changed consumer spending habits.
- Educational Services showed strong early growth from 2001 to 2006 followed by fluctuations. The industry has shown a consistent decline after 2019, with negative growth in 2021 (-0.86%) and 2022 (-1.95%). This might reflect reduced public funding, changing education models (remote learning), or demographic shifts.
- Business, Building, and Other Support Services had significant early growth from 2001 to 2006, followed by declines in 2009 (-0.52%), 2017 (-0.68%), and 2022 (-0.72%). Factors that could be at play include increase in business outsourcing, automation, and economic cycles.

```
In [87]: i = inspect(engine)
print("Tables in the database:", i.get_table_names())
```

```
Tables in the database: ['AverageWeekly', 'AverageWeekly_filter', 'Avg_week_earnings', 'Earning_Employment', 'Employment_Count', 'Final_GDP_Data', 'GDP_Data', 'LabourForce_table', 'cleaned_LabourForce', 'employment_data', 'employment_data_filtered', 'filtered_LabourForce', 'final_LabourForce', 'finalnewDataset_filled', 'geo', 'main_table', 'years']
```

Discussion

Through this project, we learned how combining multiple datasets enhances data analysis by providing deeper insights that cannot be obtained from individual datasets alone. We explored techniques for cleaning and storing big data in databases and gained hands-on experience in interacting with a server using SQL.

Key skills acquired include extracting relevant data by removing redundancy and irrelevant information, handling null values, and mapping subcategories into main categories (in our case, industries). We also learned and applied different dataset merging techniques, dealing with varied shapes and join types.

Additionally, we developed a strong understanding of SQL queries for grouping, aggregation, and ranking. We learned how to integrate Pandas with SQL for efficient data manipulation and performed various database operations. Finally, we applied data visualization techniques using Matplotlib, Seaborn, and other tools to present insights effectively.

One of the challenges our project faced was related to data quality and retrieval. Gaps and inconsistencies in our dataset sometimes created roadblocks, as missing values and

discrepancies between aggregate categories and subcategories limited the range of research questions we could explore.

Data retrieval and table joins were another challenge. Joining our tables and retrieving data from our joins was a challenging aspect of our project which made us consider exploring in further work if graph databases like Neo4j or document stores like MongoDB could simplify our queries and streamline our analysis. We are also considering to include datasets for part-time and full-time wages for further research.

Additionally, our work featured some repeated code, making our workflow more time-consuming than necessary. We are considering exploring a more modular approach in future work, such as using stored functions, to improve efficiency and maintainability.

Conclusion

Alberta's labor market shows some distinct patterns across industries, influenced by gender, age, employment type, and macroeconomic shifts. Our analysis highlights disparities in workforce composition, employment trends over time, and the relationship between wages, employment levels, and industry performance.

Employment distribution across industries remains highly gendered. Men dominate sectors like Construction (9,822.8K), Wholesale and Retail Trade (8,509.2K), and Mining, Quarrying, and Oil & Gas Extraction (5,396.1K), while women lead in Healthcare and Social Assistance (9,553.8K) and Wholesale and Retail Trade (7,803K). Despite Wholesale and Retail Trade employing both genders in high numbers compared to other industries, the industry maintains a male-biased disparity.

Female employment has grown significantly in several traditionally male-dominated industries. Between 2001 and 2024, Construction (147.15%), Healthcare (116.55%), and Public Administration (101.63%) saw the highest percentage growth in female participation. However, agriculture remains an anomaly. Female employment in the sector.

Reference

1. Ian. (2021, November 7). 3 Ways to Separate the Year, Month, and Day from a Date in MariaDB. <https://database.guide/3-ways-to-separate-the-year-month-and-day-from-a-date-in-mariadb/>
2. Open Source Database (RDBMS) for the enterprise | MariaDB. (n.d.). MariaDB. https://mariadb.com/docs/skysql-dbaas/ref/xpand/functions/STR_TO_DATE/
3. MariaDB Tutorial. (2020, April 11). MariaDB Update Statement by practical examples. <https://www.mariadbtutorial.com/mariadb-basics/mariadb-update/>
4. MySQL updating a column's values with multiple conditions - <https://stackoverflow.com/questions/64620440/mysql-updating-a-columns-values-with-multiple-conditions>

5. Adding and changing data in MariaDB. (n.d.). MariaDB KnowledgeBase.
<https://mariadb.com/kb/en/adding-and-changing-data-in-mariadb/>
6. CASE Statement - MariaDB Knowledge Base - <https://mariadb.com/kb/en/case-statement/>
7. Python and SQL Core - using INNER JOIN with Python and SQLite from scratch -
<https://www.youtube.com/watch?v=X3RM39Sucm4>
8. SQL Examples. https://www.w3schools.com/sql/sql_examples.asp
9. SQL Query Examples and Tutorial, DataCamp -
<https://www.datacamp.com/tutorial/sql-query-examples-and-tutorial>
10. Swain, Abhipsa. "How to Rename Column Name in SQL?" Scaler Topics, 4 Aug. 2022,
<https://www.scaler.com/topics/rename-column-name-in-sql/>
11. SQL Not Equal To - Syntax, Use Cases, and Examples. <https://hightouch.com/sql-dictionary/sql-not-equal-to>
12. YouTube. https://www.youtube.com/watch?v=d63LPGO_Avs
13. Mercatante, Steven. "How to Remove Square Brackets and Anything between Them with a Regex?" Stack Overflow, 13 May 2019, <https://stackoverflow.com/q/2359921>
14. Pandas Documentation — Pandas 2.2.3 Documentation.
<http://pandas.pydata.org/docs/>
15. Erkec, Esat. "Working with SQL NULL Values." SQL Shack - Articles about Database Auditing, Server Performance, Data Recovery, and More, 19 May 2021,
<https://www.sqlshack.com/working-with-sql-null-values/>
16. Working with Engines and Connections — SQLAlchemy 2.0 Documentation.
<https://docs.sqlalchemy.org/en/20/core/connections.html>
17. "MariaDB GROUP BY Clause." GeeksforGeeks, 25 Dec. 2023,
<https://www.geeksforgeeks.org/mariadb-group-by-clause/>.
18. Forrester, R. (2024, September 26). SQL Conditional Joins: Practical Guide. Five.
<https://five.co/blog/sql-conditional-joins/>
19. Grouped Bar Charts with Labels in Matplotlib. (2019, March 26).
<https://www.pythoncharts.com/matplotlib/grouped-bar-charts-matplotlib/>
20. Payne, R. (2023b, March 9). Working with NULL Values in SQL. Database Journal.
<https://www.databasejournal.com/features/sql-null-values/#:~:text=The%20SQL%20COALESCE%20and%20IFNULL,it%20returns%20the%20>
21. SQLAlchemy - Getting a list of tables. (n.d.). Stack Overflow.
<https://stackoverflow.com/questions/6473925/sqlalchemy-getting-a-list-of-tables>
22. W3Schools.com. (n.d.). https://www.w3schools.com/sql/sql_like.asp
23. Using cast function and coalesce function to generate a fixed width file format. (n.d.). Stack Overflow. <https://stackoverflow.com/questions/42426285/using-cast-function-and-coalesce-function-to-generate-a-fixed-width-file-format>
24. GeeksforGeeks. (2022, February 28). How to Execute raw SQL in SQLAlchemy. GeeksforGeeks. <https://www.geeksforgeeks.org/how-to-execute-raw-sql-in-sqlalchemy/>
25. SQLAlchemy. (n.d.). Can `sqlalchemy.inspect(engine).get_columns()` return values already converted to Python types without previous definition of table in `sqlalchemy.Table`? · sqlalchemy/sqlalchemy · Discussion #11689. GitHub.
<https://github.com/sqlalchemy/sqlalchemy/discussions/11689>

